

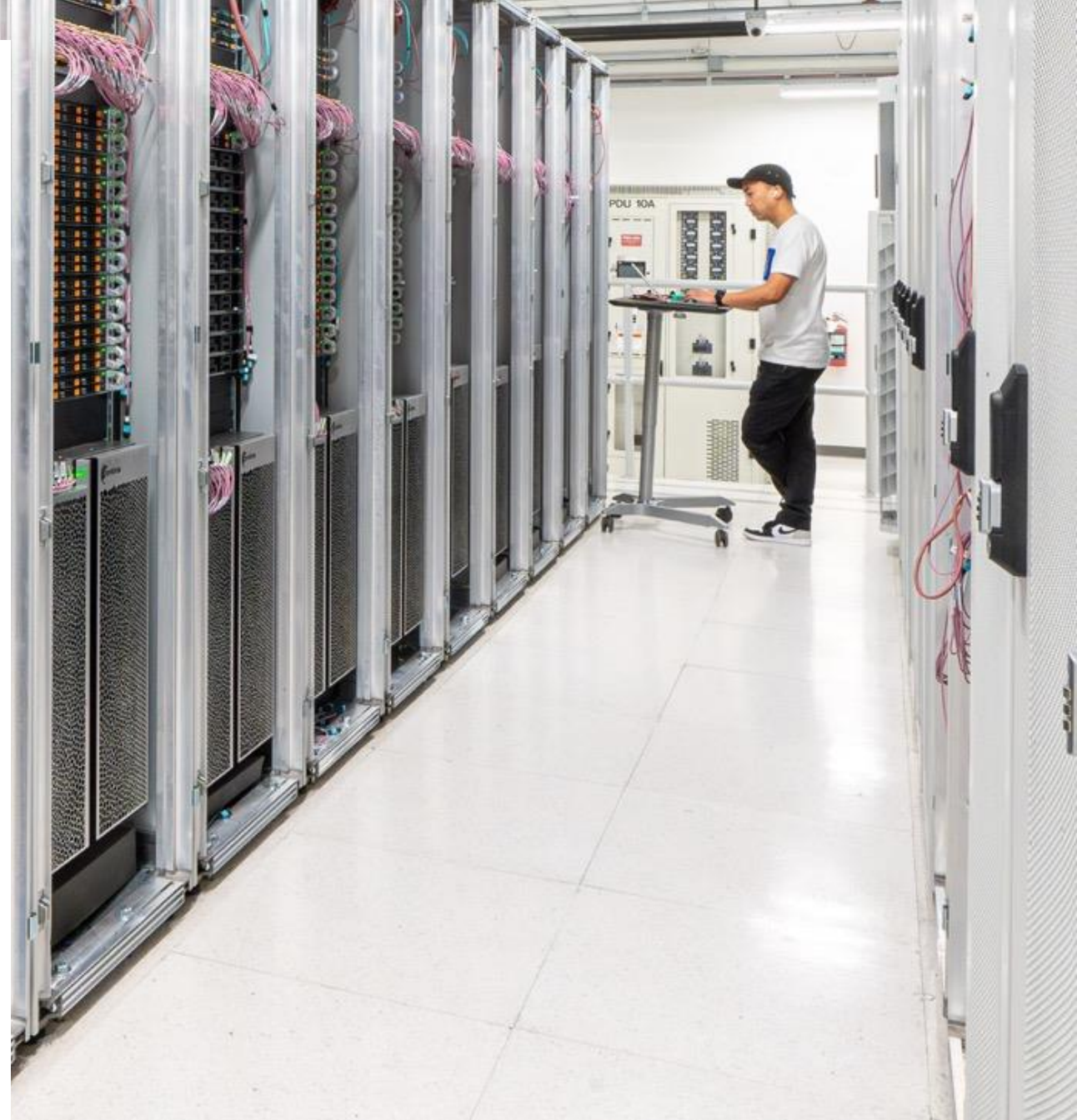
Overclocking Training, Inference, and HPC with Cerebras

Sylvia Howland, Cerebras Customer Success

**ISC High Performance 2026
June 22nd, Hamburg, Germany**

Agenda

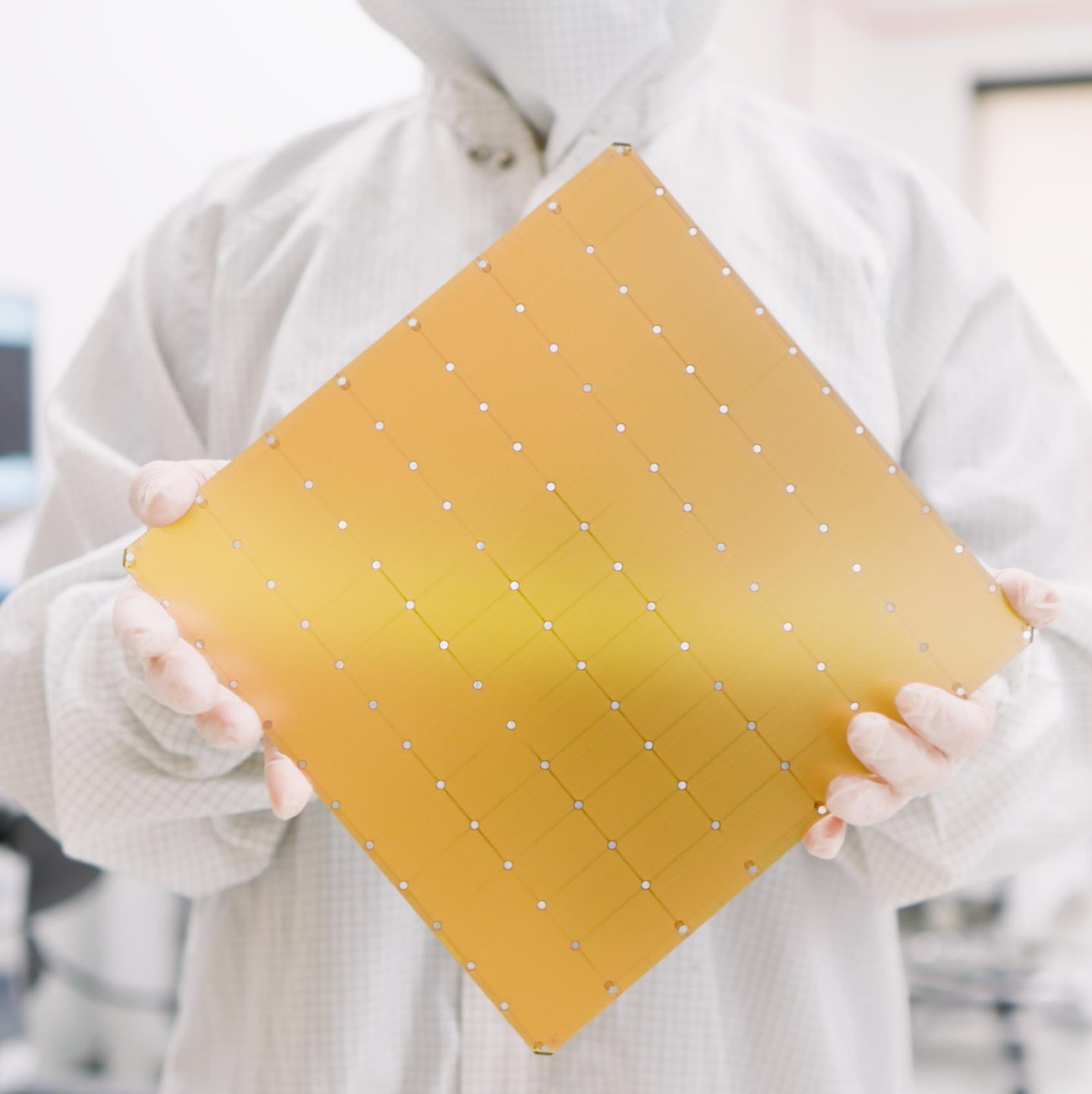
- Intro to Wafer-Scale Computing
 - *Engines, Clusters, and You*
- The Cerebras Training Stack
 - *From PyTorch to Bare Steel*
- Cerebras Inference
 - *Powering Instant AI*
- The Cerebras SDK
 - *Empowering HPC Applications*





Intro to Wafer Scale Computing

Engines, Clusters, and You



Cerebras Wafer Scale Engine 3 (WSE-3)

The fastest AI chip on earth **again**

4 trillion transistors

46,225 mm² silicon

900,000 cores optimized for sparse linear algebra

5nm TSMC process

125 Petaflops of AI compute

44 Gigabytes of on-chip memory

21 PByte/s memory bandwidth

214 Pbit/s fabric bandwidth

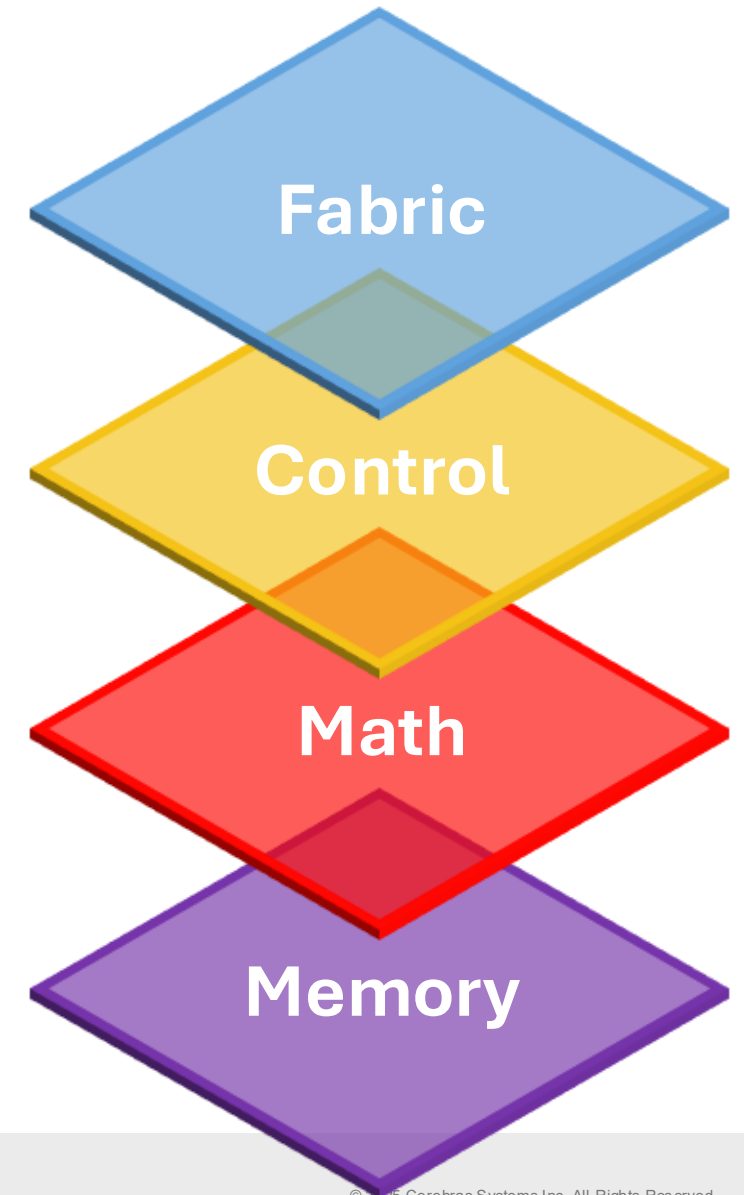
Completely Uniform Mesh

Numerical efficiency is all about bandwidth

Programmability for yet-to-exist algorithms

Numerical algorithms are the core of ML & HPC

Storage for activations and parameters



All key components brought together on a single tile

Fabric

Vector streaming engine
Activation selection filter
Sparse stream support

Control

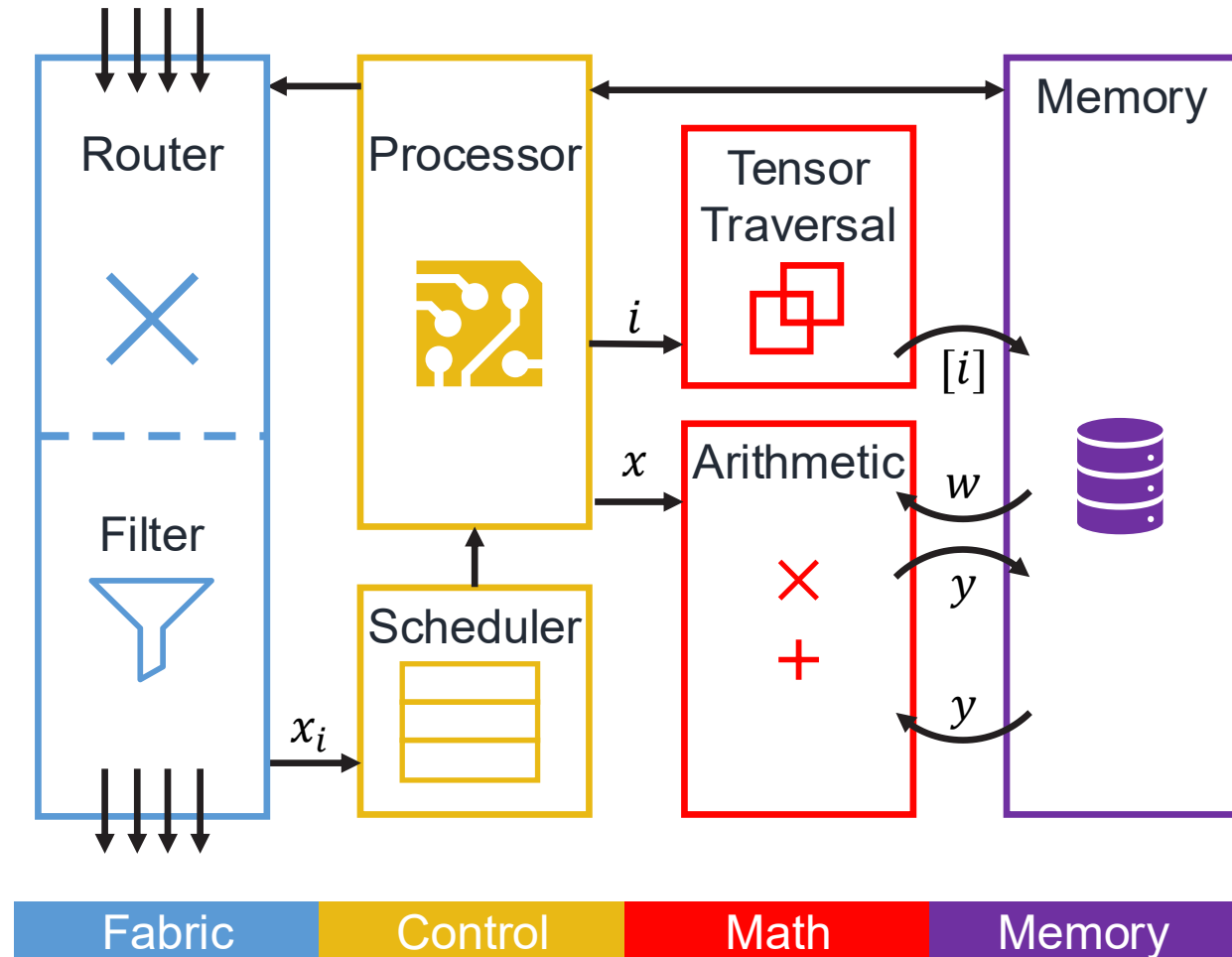
Dataflow scheduling
Nine execution threads

Math

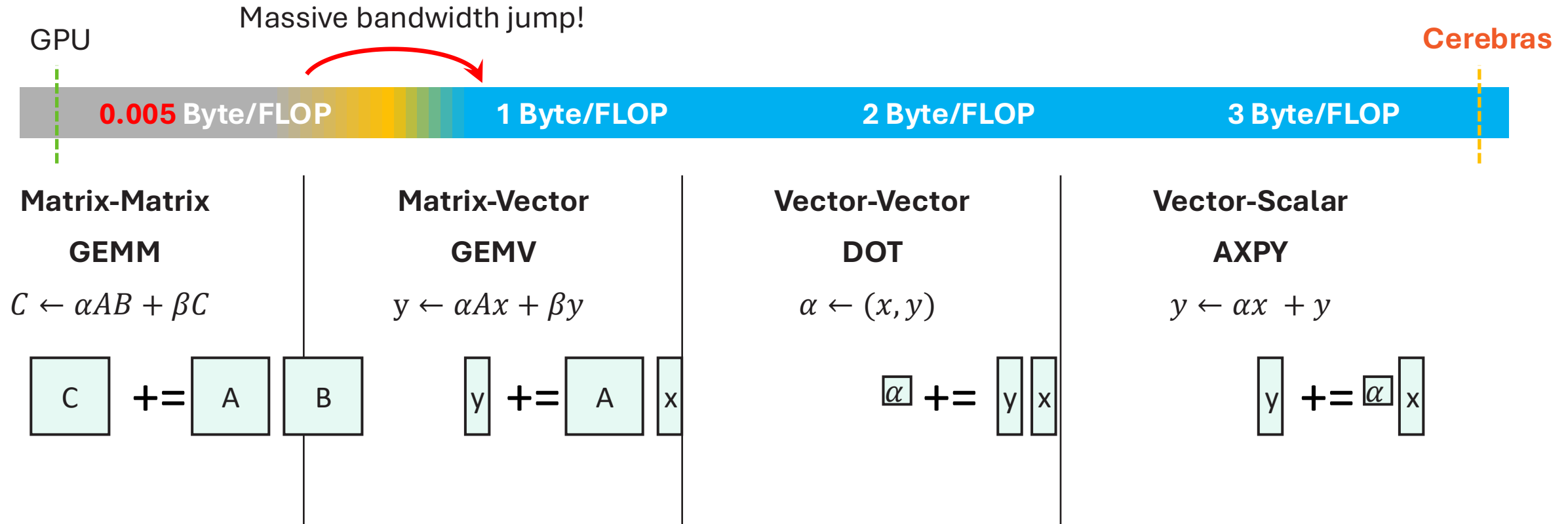
Floating point and integer
Entropy engine
Tensor access patterns

Memory

Single cycle access latency
Eight Reads + Four Writes

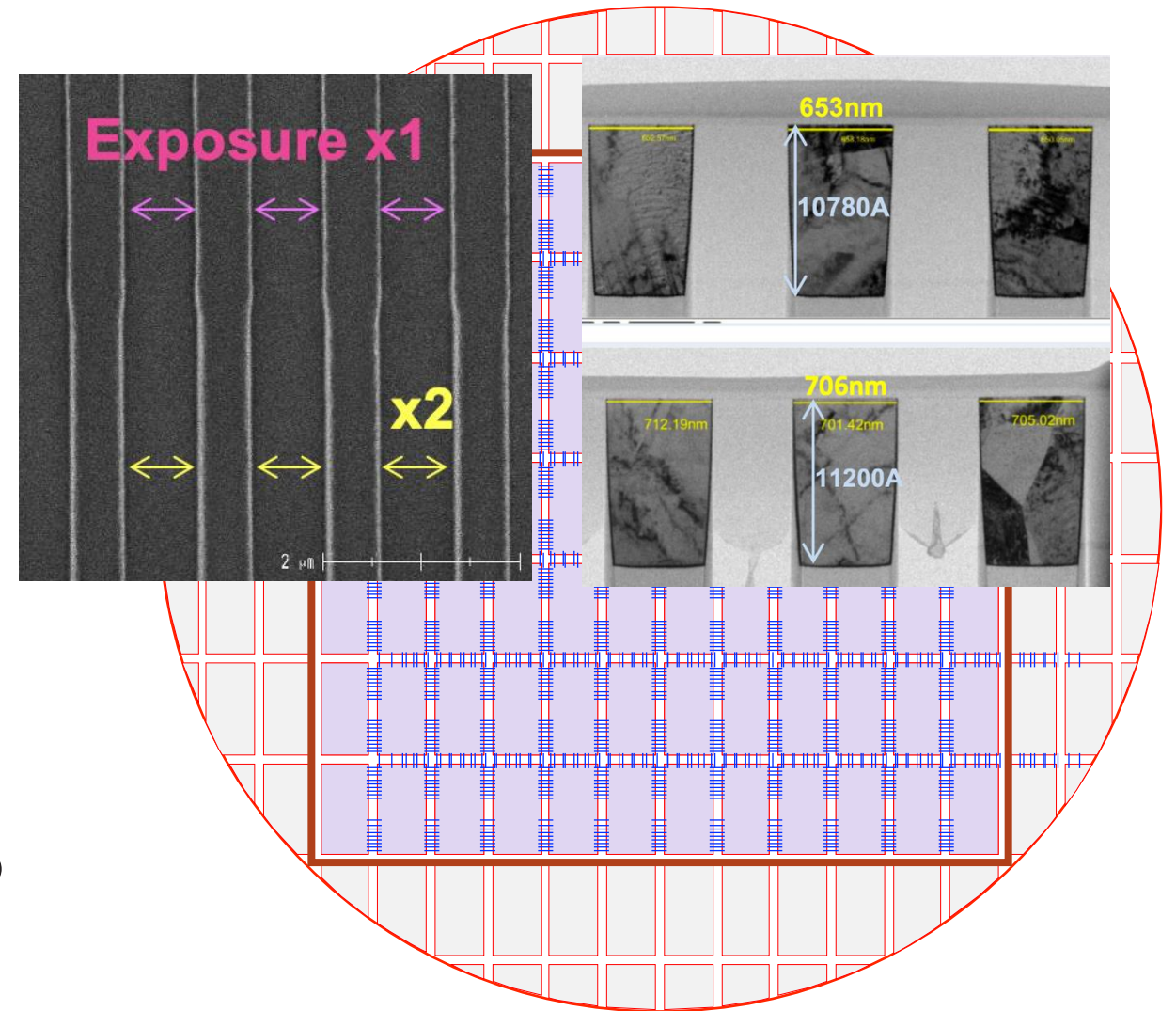


Full memory performance at all BLAS Levels



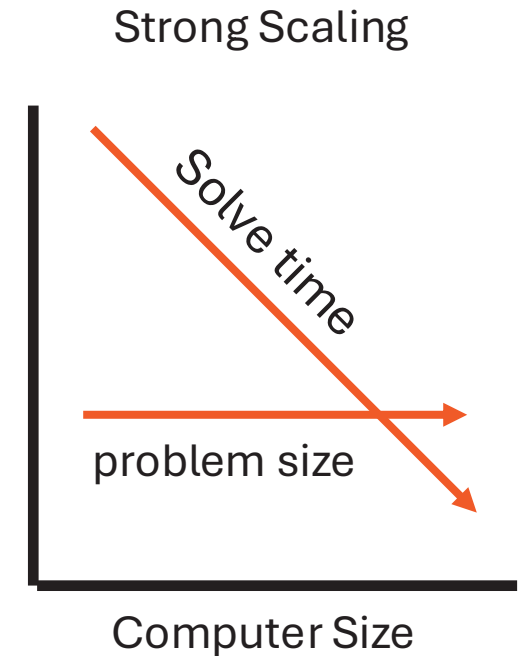
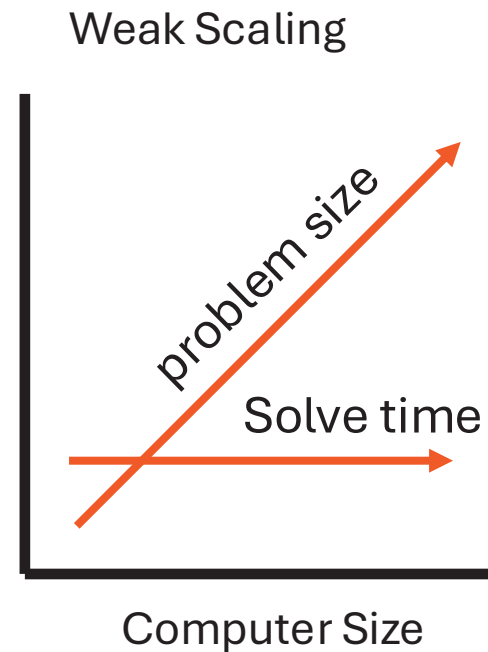
Uniform Fabric Across Entire Wafer

- Designed to scale beyond individual die
- Bridge <1mm across scribe lines between die
- Source synchronous parallel interface
- Redundancy with training and auto-correction state machine
- Uniform bandwidth across entire wafer
- The entire wafer is a single chip all with on-chip bandwidth
- Full bandwidth within die and between die
- Wafer integration enables ultra short inter-chip links

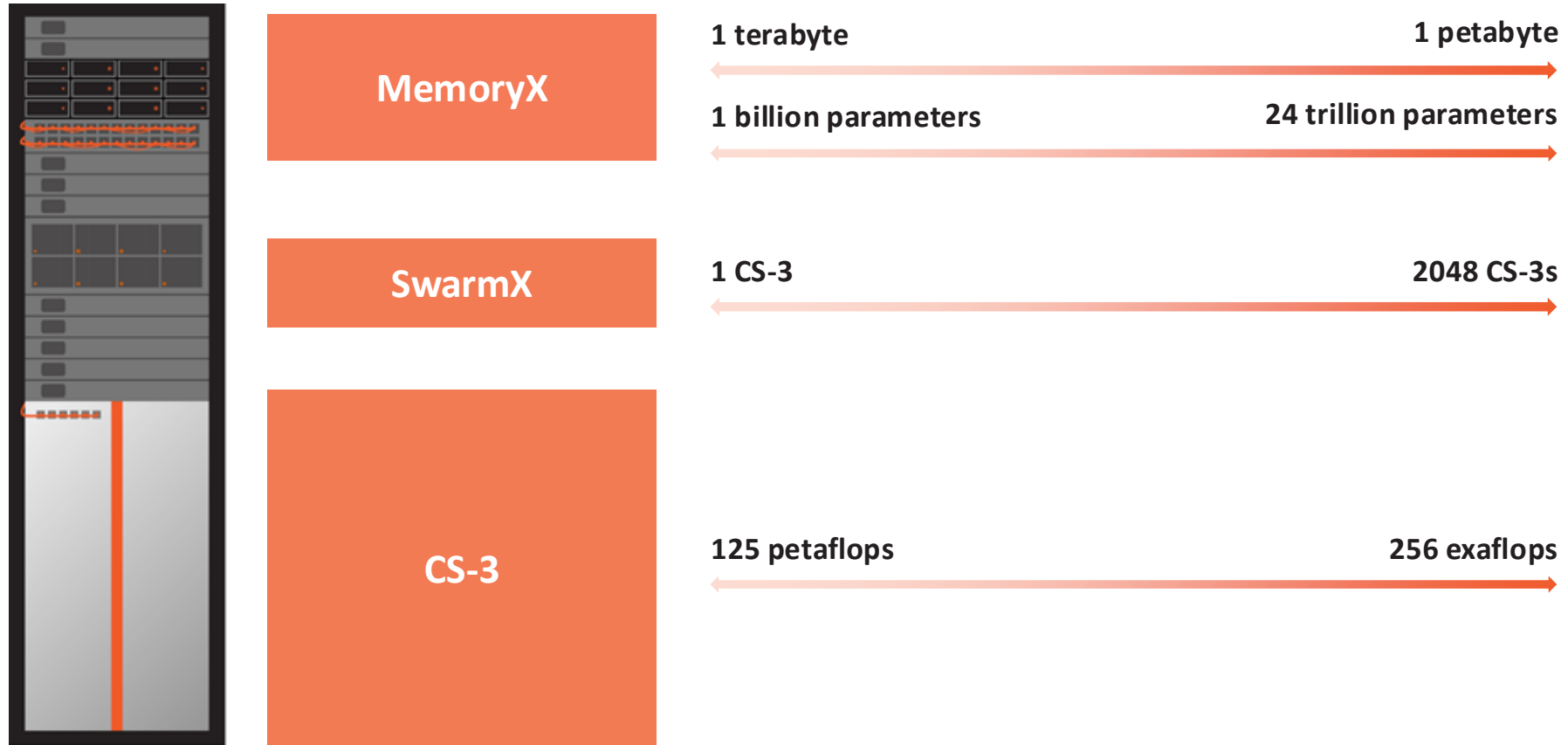


Designed for Strong Scaling

- Contemporary platforms fail to Strong Scale
- Must spread workload thin
- Eliminate bulk processing
- Fine-grain messaging among cores
- Offload bookkeeping overheads
- Fabric support for communication patterns



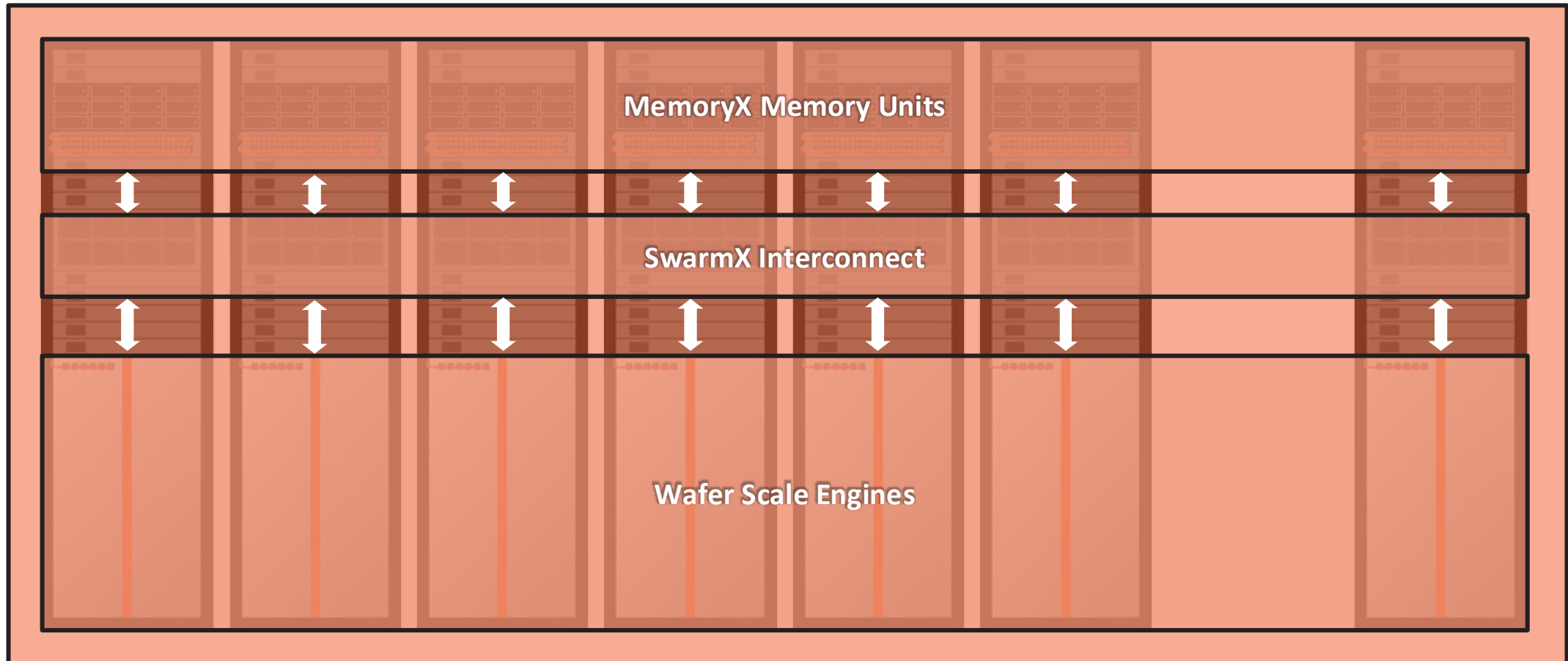
Wafer Scale Cluster: Scalable AI Supercomputer



Exa-scale Performance

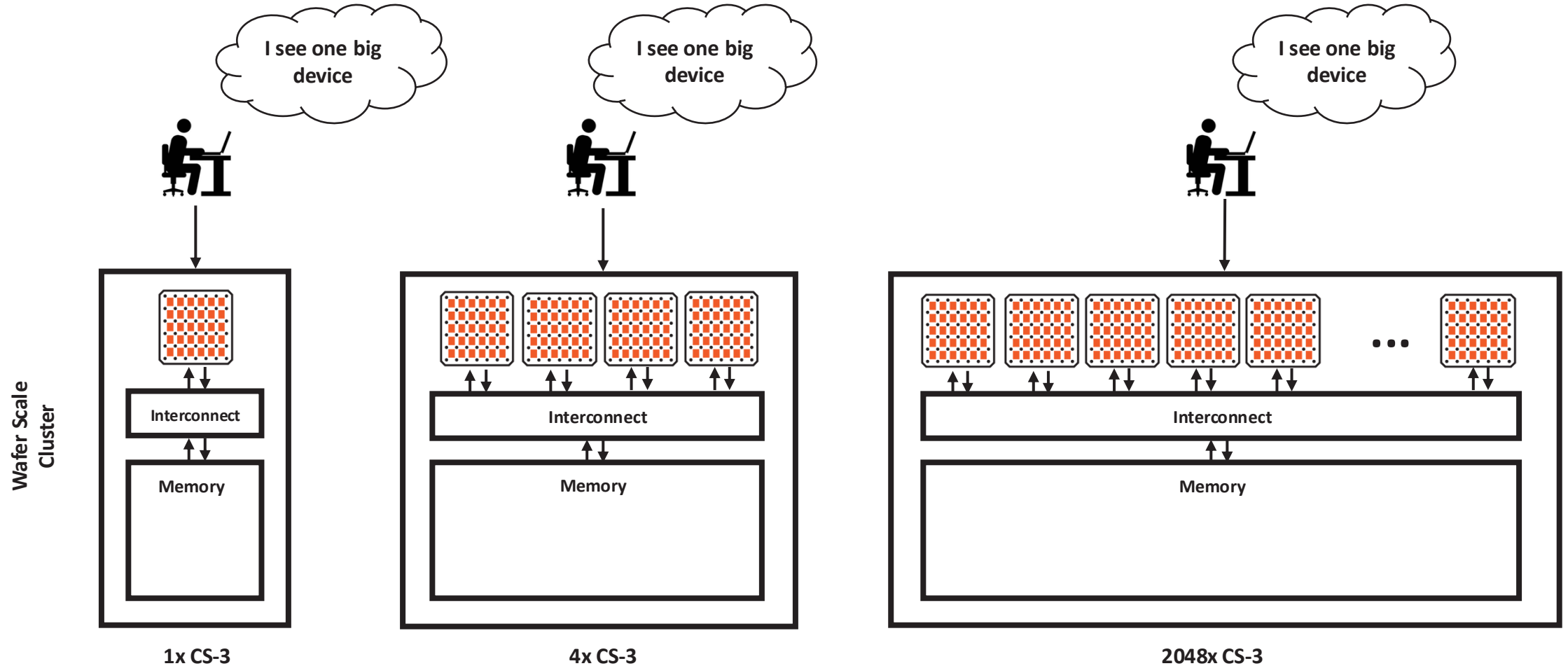


Single Device Simplicity

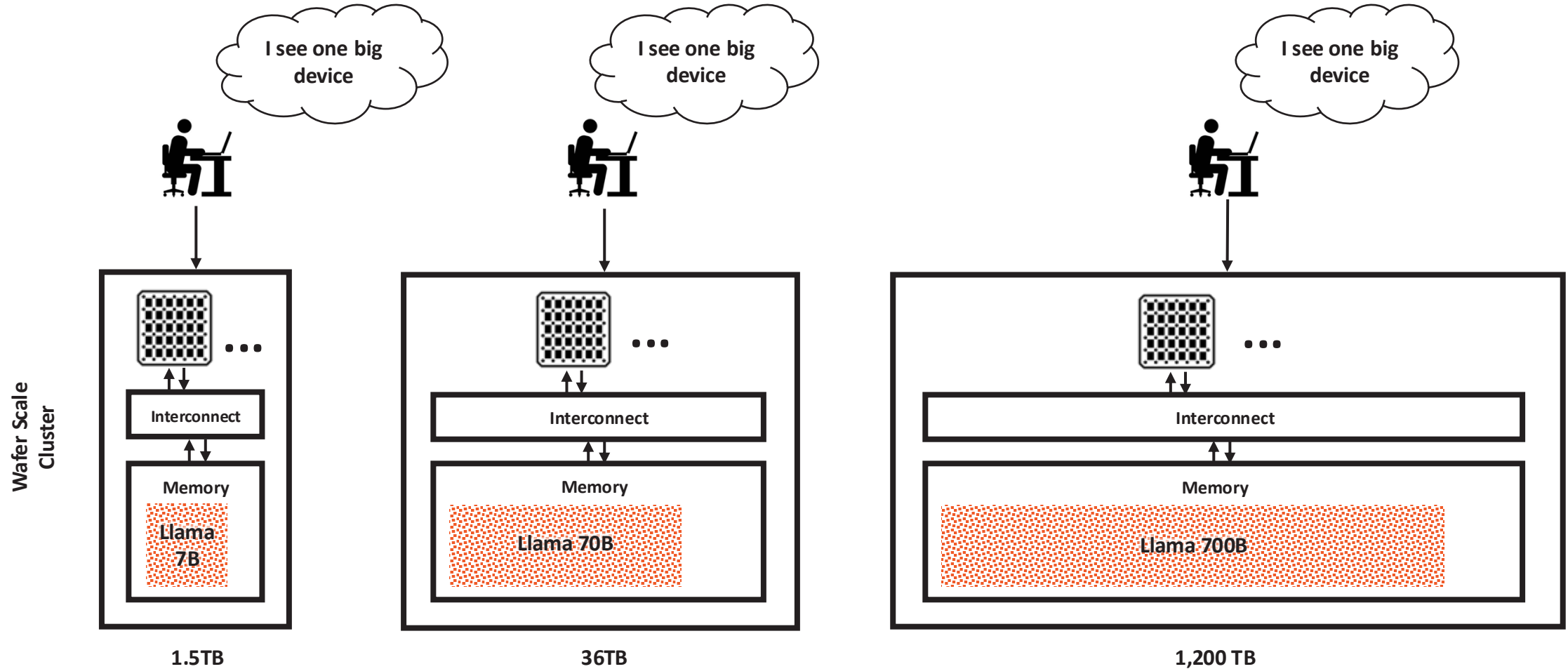


1 to 2048 CS-3s Look and Program Like a Single Device

You Program It Like A Single Device No Matter The Cluster Size

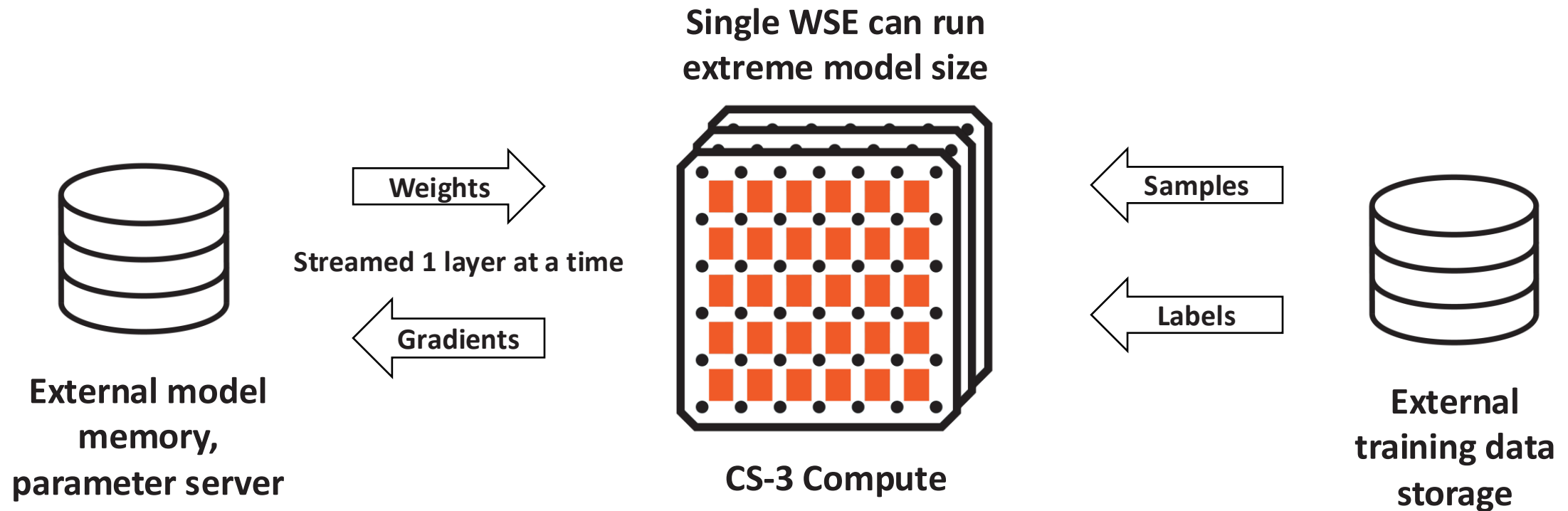


And Your Model Always Fits 1B or 1T Parameters



Weight Streaming Technology

Disaggregates Memory and Compute for Trillion Parameter Model Training



Scale model size and training speed independently

MemoryX External Memory

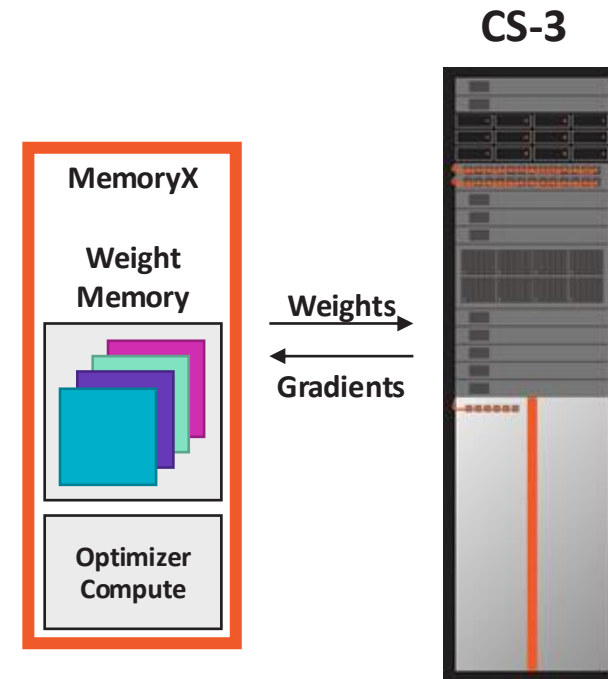
Virtually Unlimited Model Weight Capacity

Model capacity not limited by device

- Weights streamed onto wafer to compute layer
- Weights trigger compute using HW dataflow
- Weights are never stored on wafer

Decoupling weight optimizer compute

- Gradients streamed out of wafer
- Weight update occurs in MemoryX



Memory hierarchy capable of massive models on single device

SwarmX Fabric

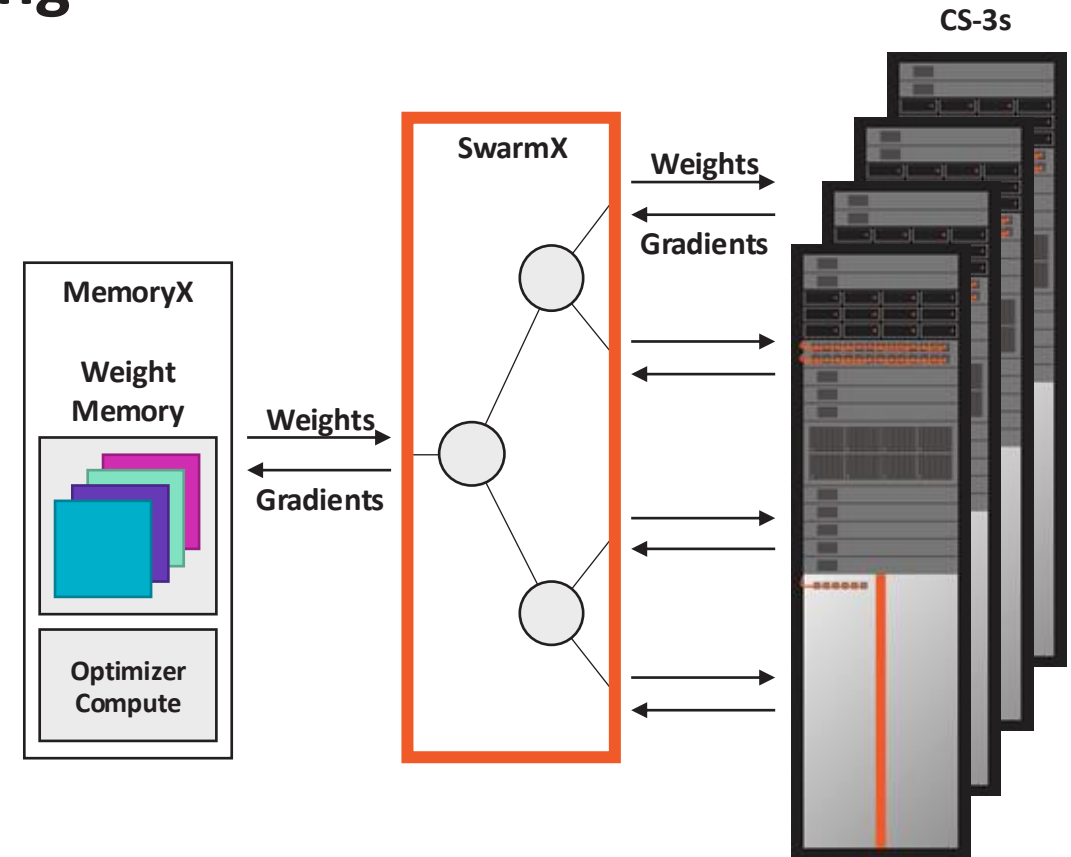
Purpose Built Interconnect for Simple Scaling

Data-parallel only training across CS-3s

- Weights are broadcast to all CS-3s
- Gradients are reduced on way back

Multi-system scaling with the same execution model as single system

- Same system architecture
- Same network execution flow
- Same software user interface



Scaling to cluster compute while operating like a single device

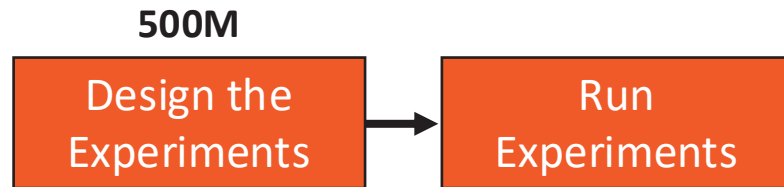
Why does this matter?

How to get good model quality at scale

500M

Design the
Experiments

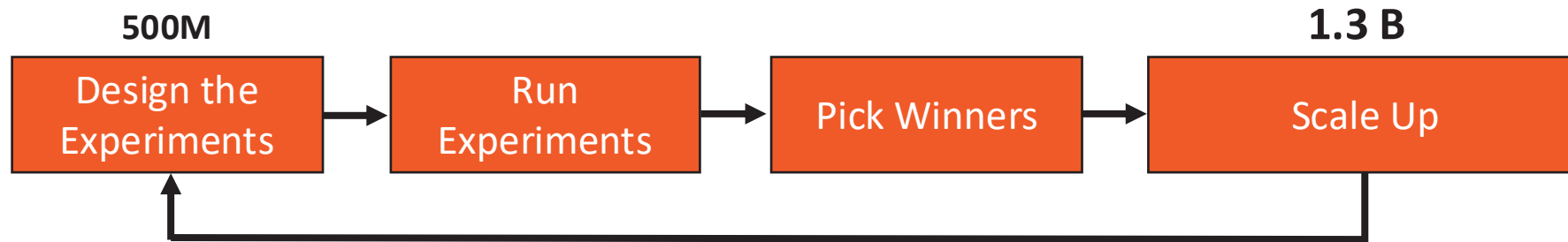
How to get good model quality at scale



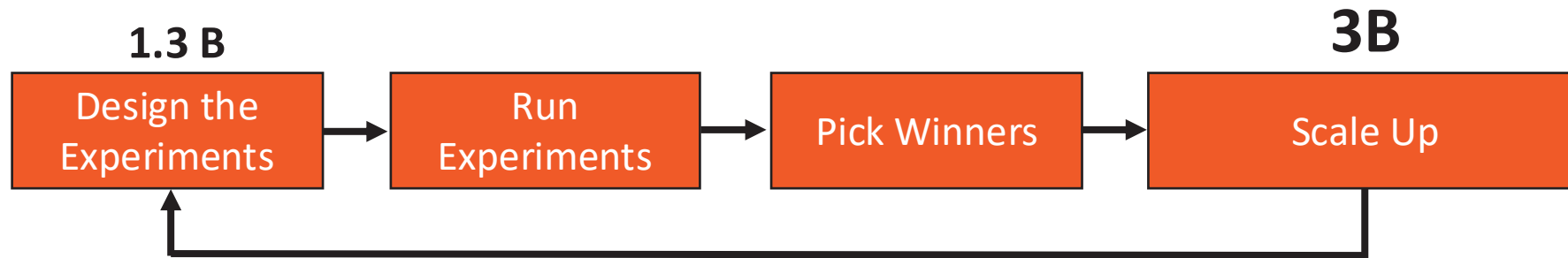
How to get good model quality at scale



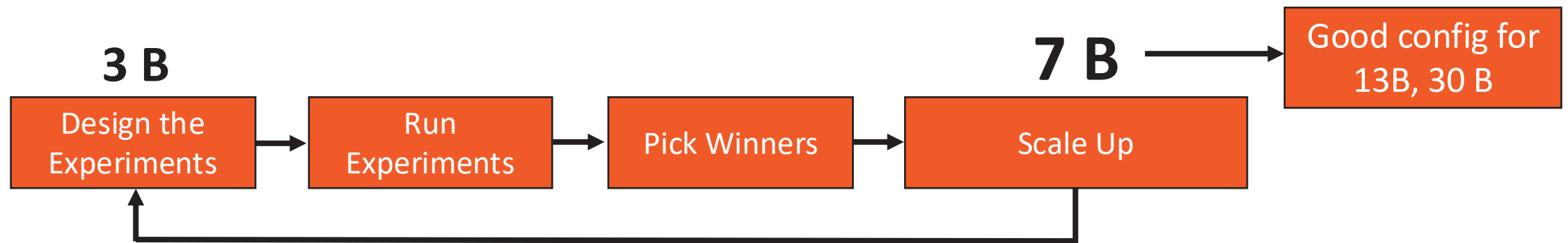
How to get good model quality at scale



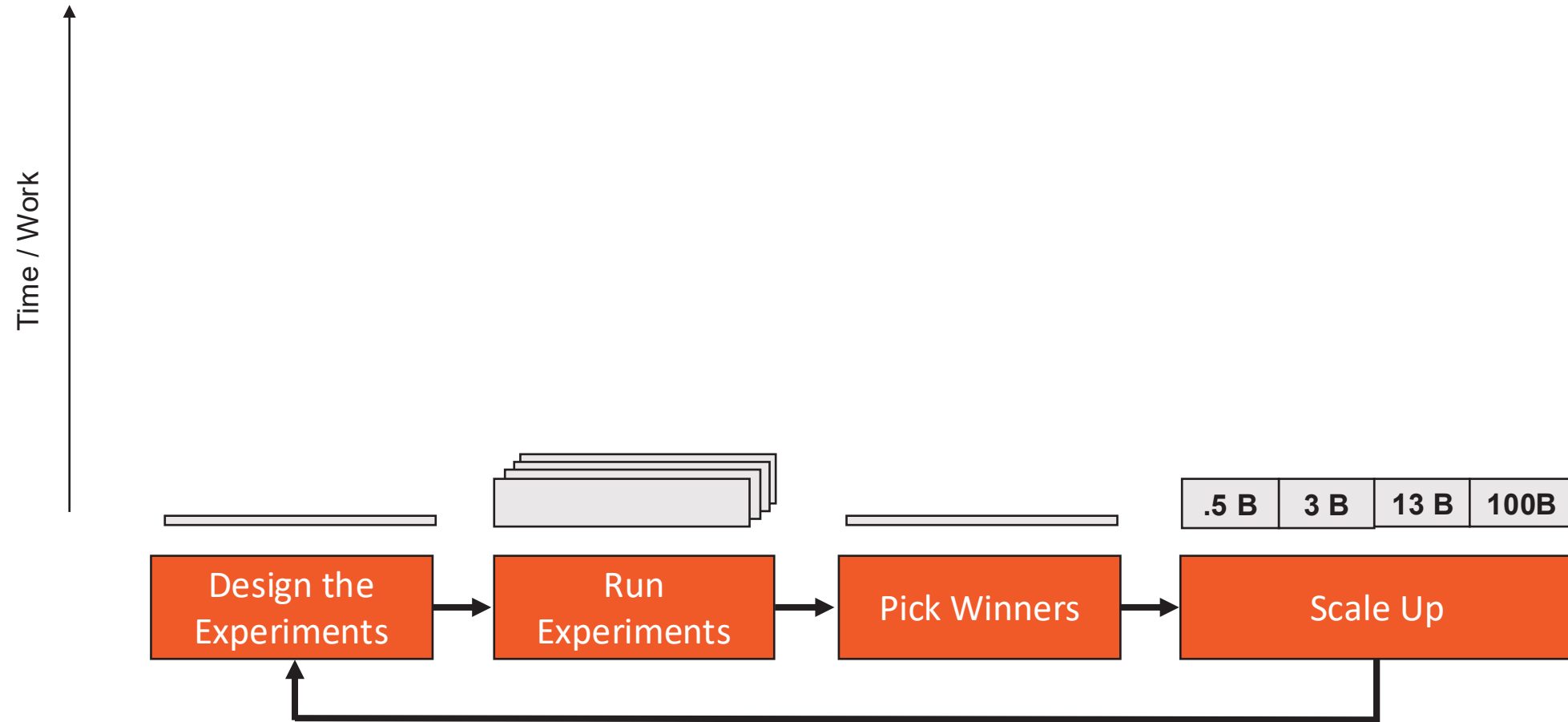
How to get good model quality at scale



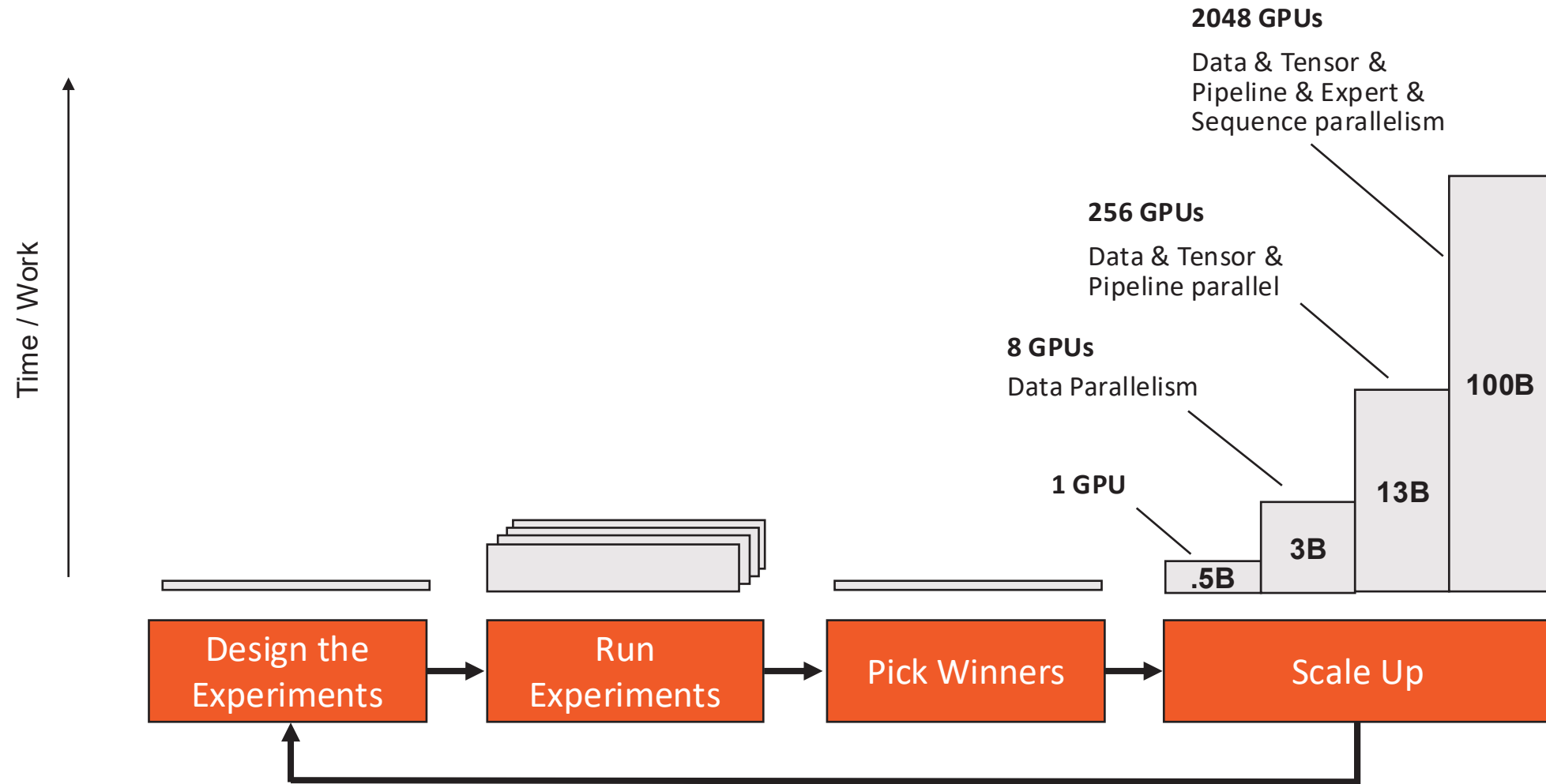
How to get good model quality at scale



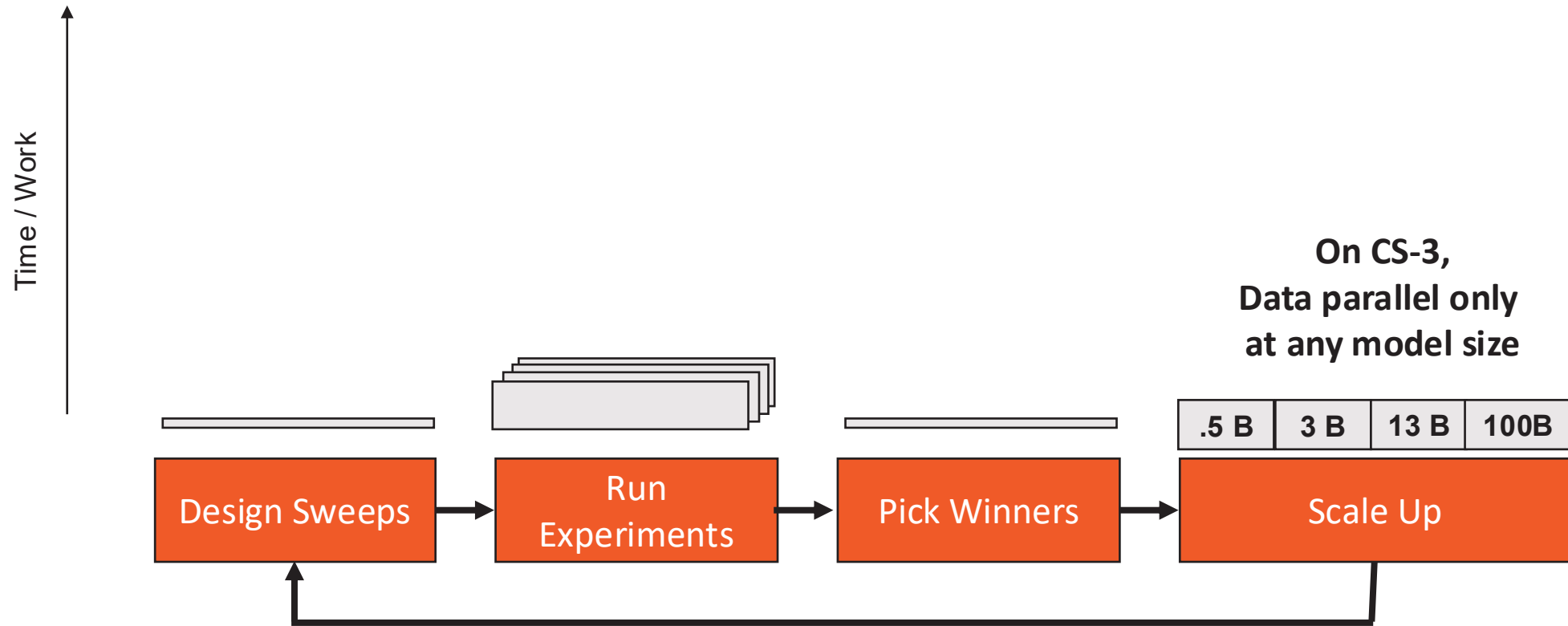
How to get good model quality at scale



How to get good model quality at scale (on GPUs)



Cerebras gets you to high-quality large models faster & more cheaply



Open-sourced models trained with Cerebras



BTLM-3B-8K

3B Parameters • 8K Context
7B Performance in a 3B Model

Open Source. Trained on Cerebras



CrystalCoder

7B Parameters • 1.3T Tokens
Coding + English. The most open source
& reproducible model in the world.

Open Source. Trained on Cerebras



Jais

13B & 30B
State of the art Arabic +
English models

Open Weights. Trained on Cerebras



Cerebras-LLaVA

7B & 13B
Visual Question & Answering

Trained on Cerebras



Med42

Fined-tuned Llama2-70B
Medical Q&A LLM Scores
72% on USMLE

Trained on Cerebras

gigaGPT



GPT-3 in 565 lines of code
Cerebras implementation
of nanoGPT

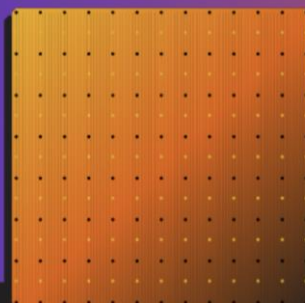
Open Source. Trained on Cerebras



SlimPajama

627B token dataset
Extensively deduplicated dataset
with twice the perf/token

Open Source



Cerebras-GPT

111M-13B Parameters
First family of GPT models
released under Apache 2.0

Open Source. Trained on Cerebras



FLOR

6.3B Parameters
State of the art Catalan +
Spanish + English model

Open Source. Trained on Cerebras



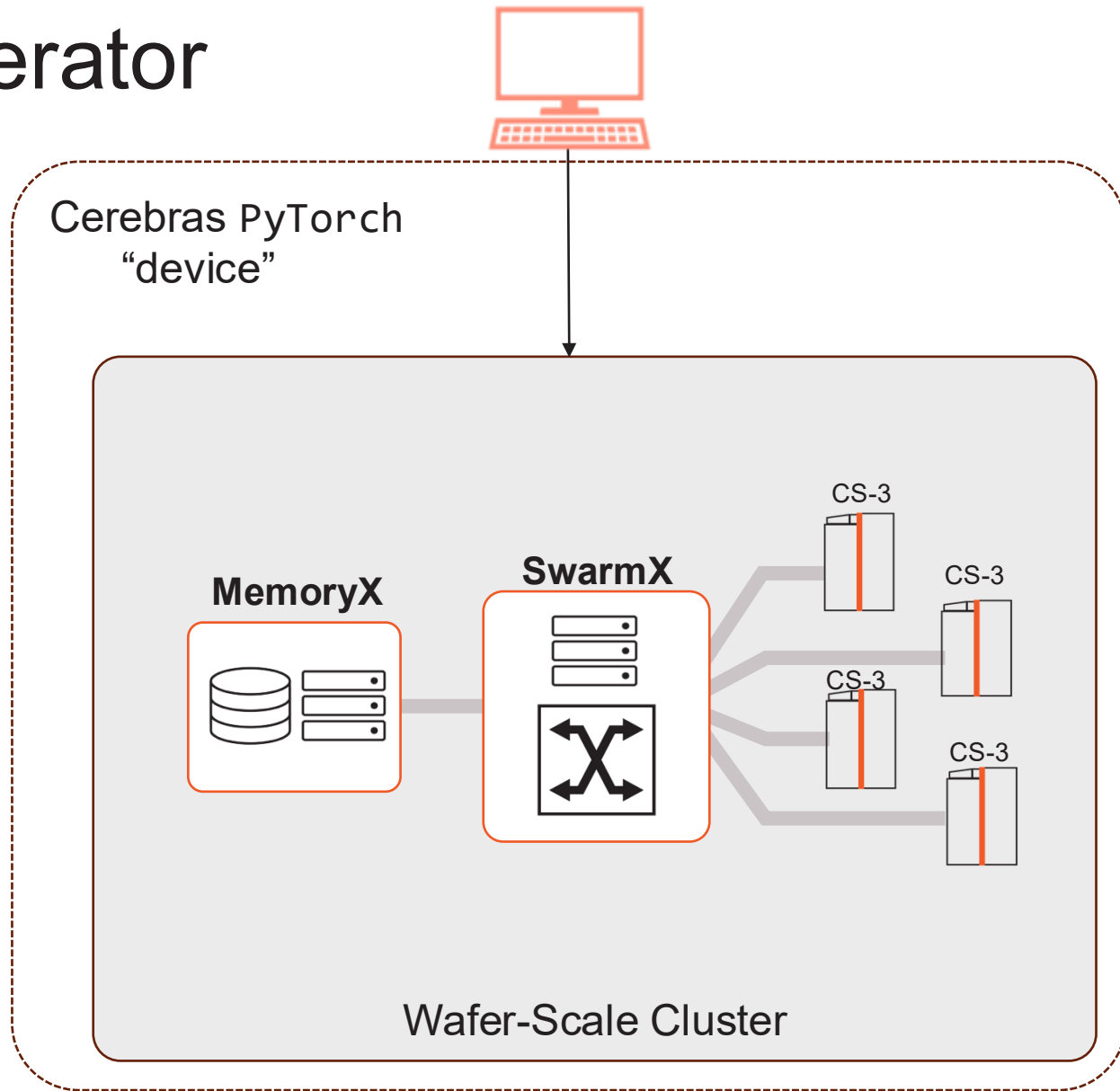
The Cerebras Training Stack

From PyTorch to Bare Steel

The *Cluster* is the ML Accelerator

Disaggregation of memory and compute

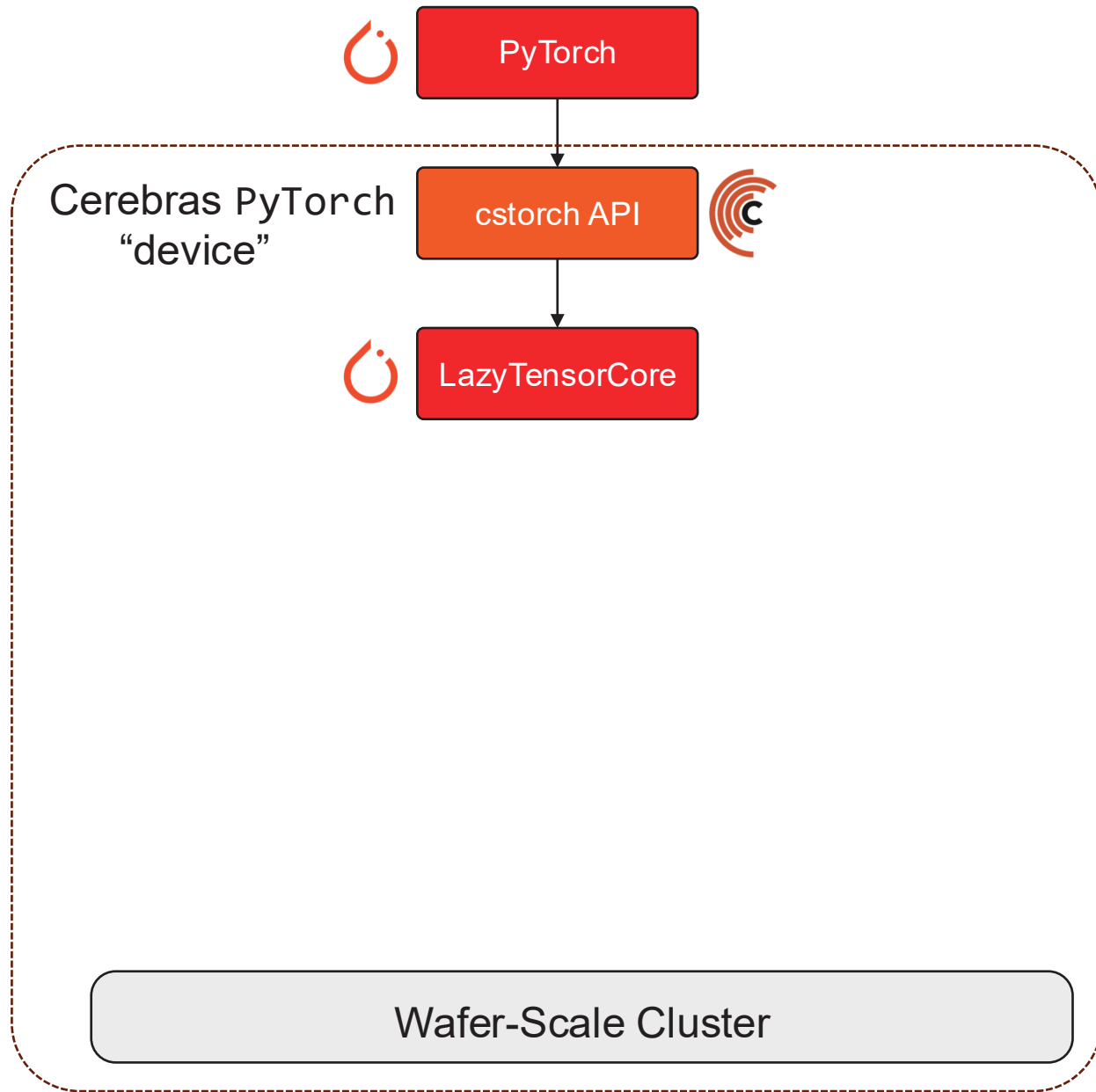
- Stores model weights off-wafer with fast streaming
- MemoryX size and compute device count can be scaled independently
- The entire cluster acts one remote accelerator
- All the complexity of distributed training, data parallelism, and scaling are handled automatically by the Wafer Scale Cluster
- Exposed to the user as a single PyTorch device!



cstorched Software Stack

Frontend API

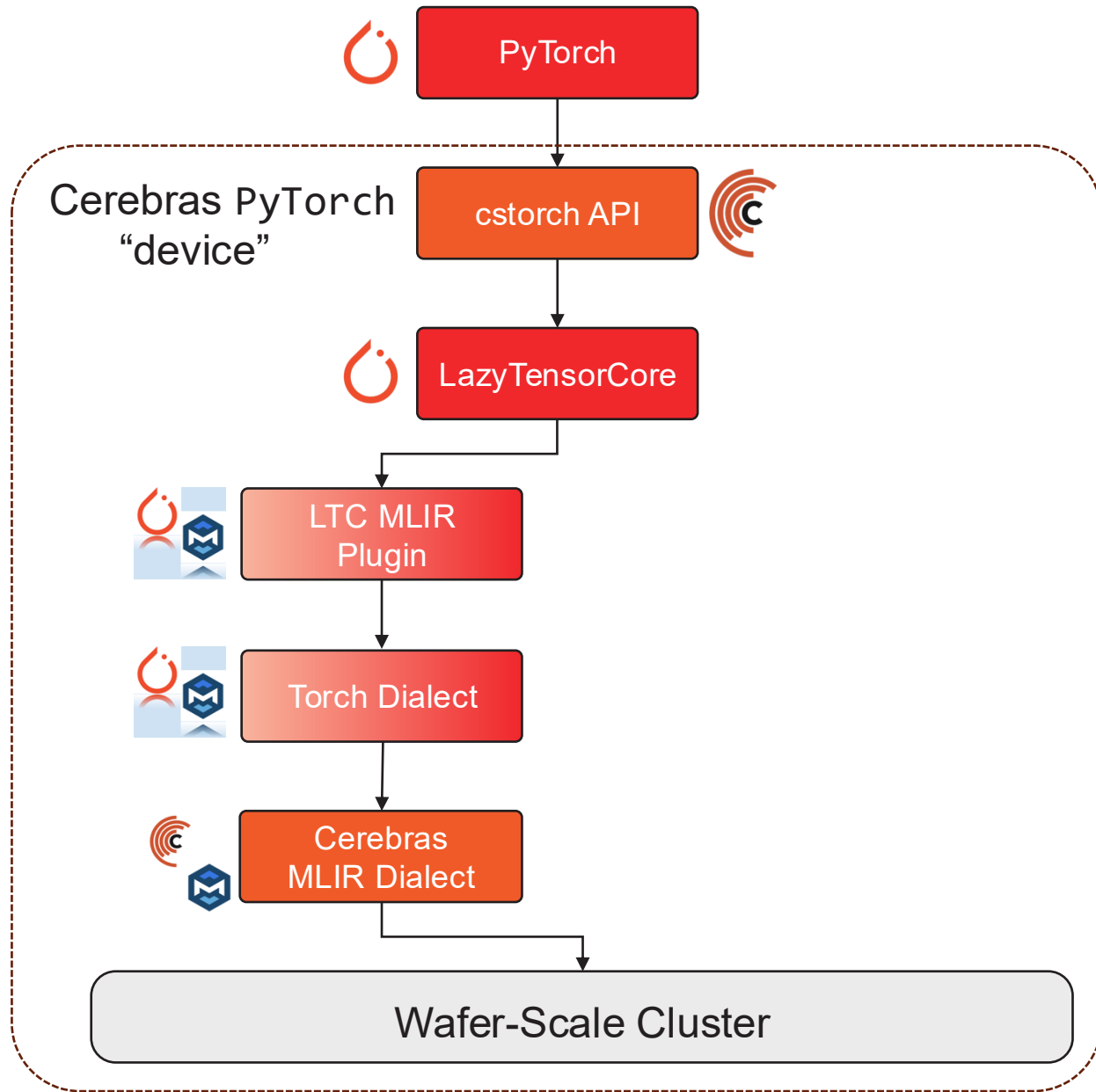
- Model code is written in vanilla PyTorch
- cstorched API mirrors PyTorch API
 - Helps with single-device abstraction
 - Used for some drop-in replacements and unique Cerebras features
- Tensor Ops traced through LazyTensorCore (LTC)
 - Graph-by-execution with lazy evaluation
 - Also drives Google's xla/tpu device



cstorch Software Stack

Compilation

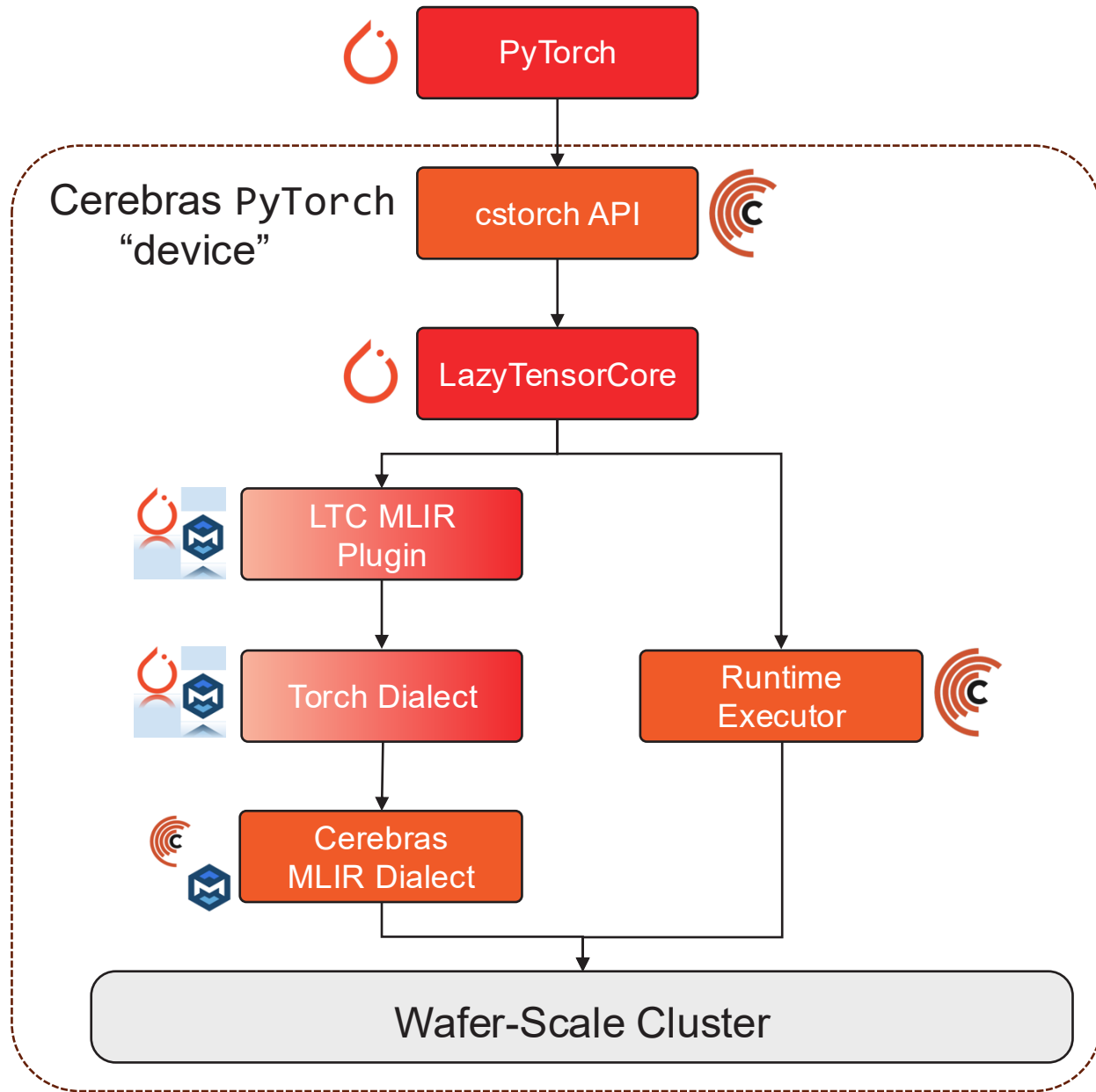
- Model code is written in vanilla PyTorch
- cstorch API mirrors PyTorch API
 - Helps with single-device abstraction
 - Used for some drop-in replacements and unique Cerebras features
- Tensor Ops traced through LazyTensorCore (LTC)
 - Graph-by-execution with lazy evaluation
 - Also drives Google's xla/tpu device
- MLIR translation from LTC provided by torch-mlir
 - Hardware-focused compiler ecosystem for Torch
- Cerebras MLIR stack handles cluster optimizations



cstorch Software Stack

Runtime Executor

- Model code is written in vanilla PyTorch
- cstorch API mirrors PyTorch API
 - Helps with single-device abstraction
 - Used for some drop-in replacements and unique Cerebras features
- Tensor Ops traced through LazyTensorCore (LTC)
 - Graph-by-execution with lazy evaluation
 - Also drives Google's xla/tpu device
- MLIR translation from LTC provided by torch-mlir
 - Hardware-focused compiler ecosystem for Torch
- Cerebras MLIR stack handles cluster optimizations
- Tensors get transferred to the cluster as needed
 - Initial weights sent to MemoryX before first step
 - Inputs are streamed each step from the custom data executor



The Cerebras Modelzoo

github.com/Cerebras/modelzoo

As a **starting** point

Reference model implementations:
GPT-3, LLaMa, Mistral, BERT, LLaVa, and much more!

Data preparation scripts

Configurations for multiple model scales

For **benchmarking**

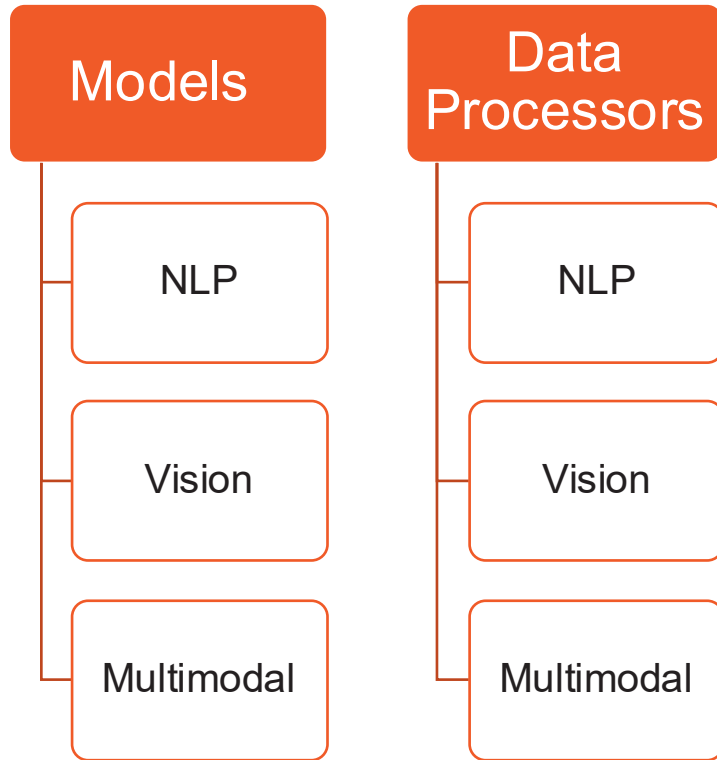
Tuned implementations for optimal performance

To **develop** new models

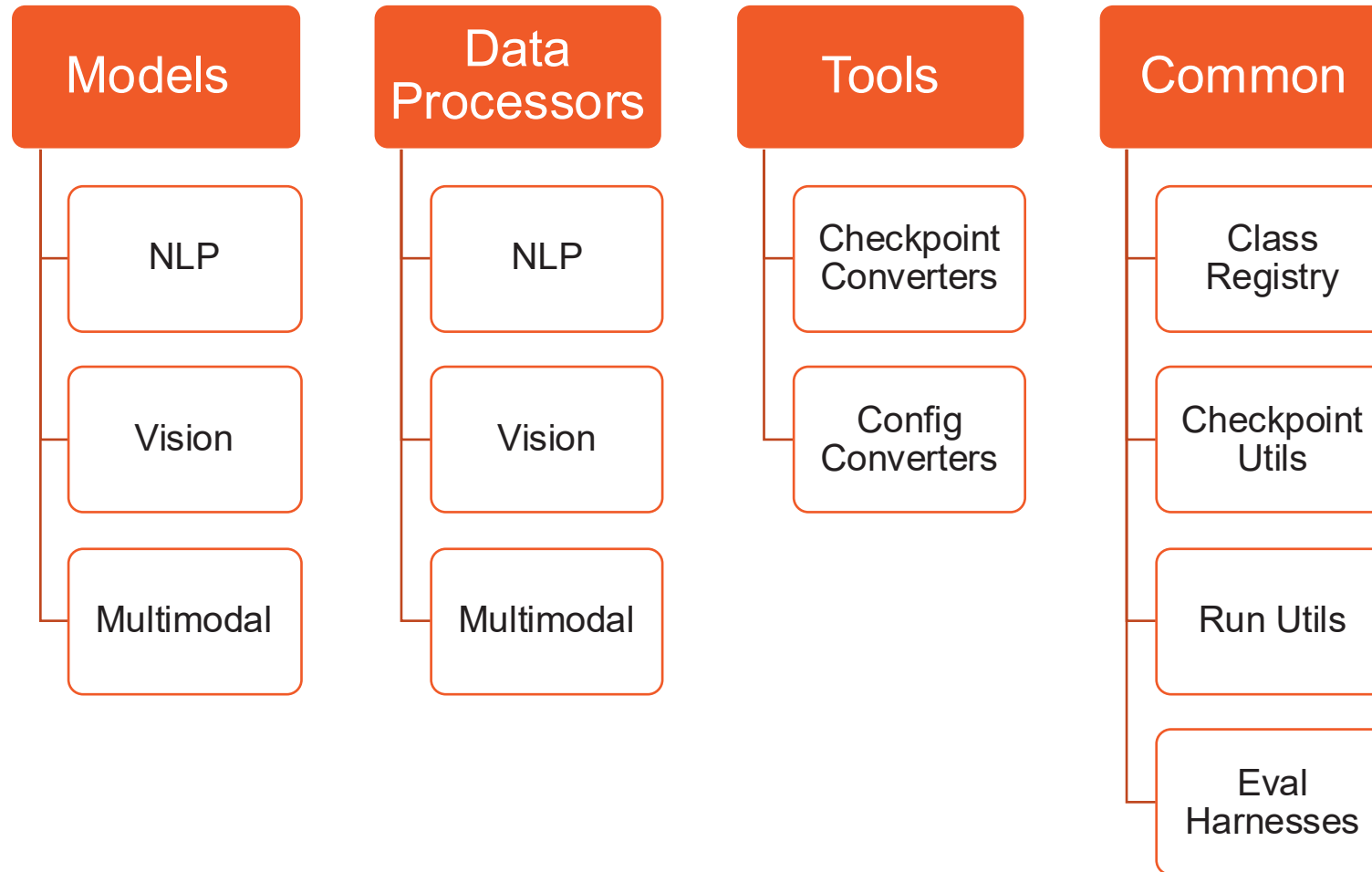
Cerebras PyTorch & Layer APIs

File organization as a template to start your own models

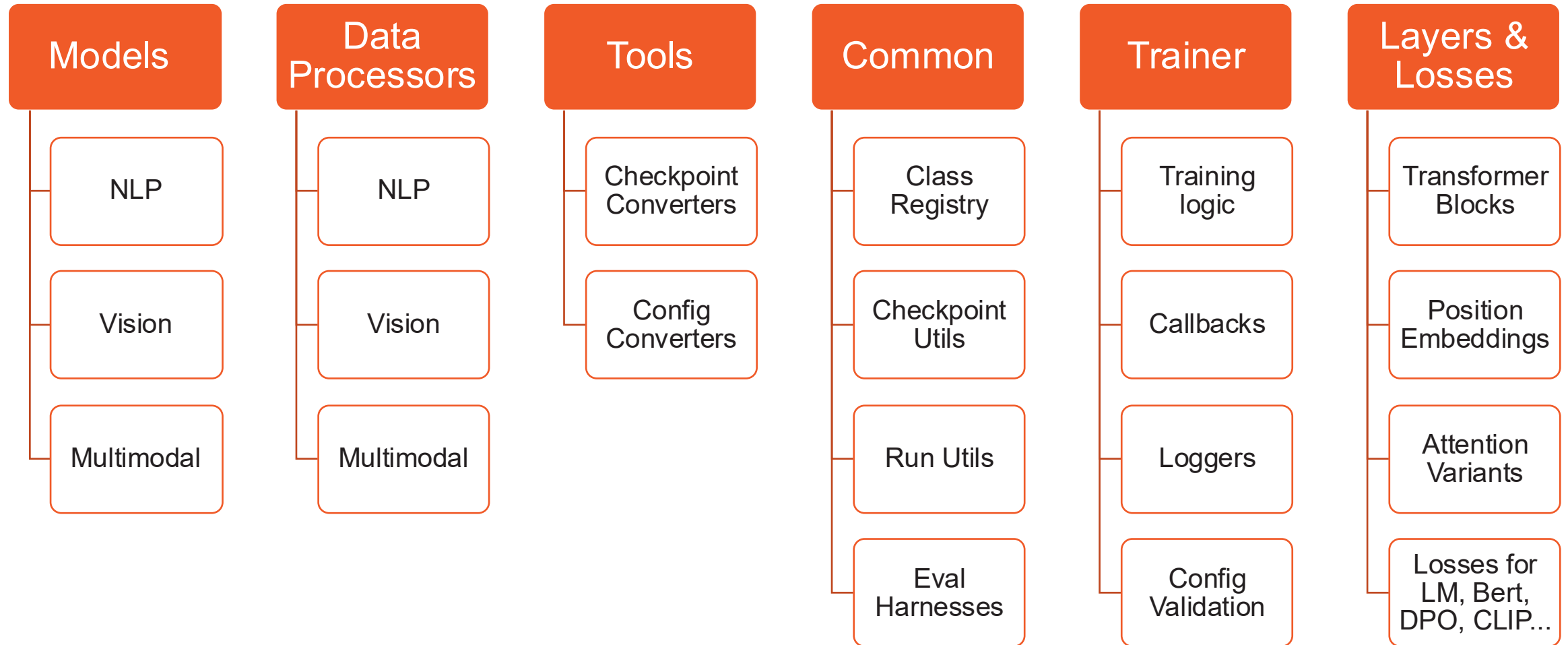
Modelzoo: Batteries Included



Modelzoo: Batteries Included



Modelzoo: Batteries Included



The Modelzoo Command-Line Interface

- The `modelzoo` library comes with an accessible command-line interface (CLI)
- Supported commands include:
 - Model training and evaluation
 - Conversion of model configs and checkpoints
 - Summarizing supported models and breaking down a given model config
 - Data preprocessing and breaking down a given `modelzoo` `DataProcessor`
- The `modelzoo` also has an LLM assistant! It can:
 - Answer questions about Cerebras' hardware, software, and documentation
 - Process natural language instructions to execute `modelzoo` CLI commands (with permission)
 - All you need is an API key from inference.cerebras.ai

Modelzoo CLI: Training a Model

```
cszoo fit pretraining_tutorial/model_config.yaml
```

Modelzoo CLI: Converting to Huggingface

```
cszoo checkpoint convert \  
--model llama \  
--src-fmt cs-auto \  
--tgt-fmt hf \  
--config pretraining_tutorial/model_config.yaml \  
--output-dir pretraining_tutorial/to_hf \  
pretraining_tutorial/model/checkpoint_0.mdl
```

Modelzoo CLI: Using the Assistant

```
cszoo assistant "How do models interface with DataProcessors?"
```

```
Allow the assistant to run command 'cszoo data_processor list'? [Y/n]: ...
```

```
Allow the assistant to run command 'cszoo data_processor info GptHDF5DataProcessor'? [Y/n]: ...
```

The GptHDF5DataProcessor is a data processor that supports multiple models, including bloom, btlm, falcon, gemma2, gpt-neox, gpt2, gpt3, gpt_backbone, gptj,jais, llama, mistral, mixtral, santacoder, and starcoder. This data processor is used to preprocess and prepare data for these models, which are primarily used for natural language processing tasks.

To understand how models interface with DataProcessors, we can see that each model has a specific data processor associated with it. In this case, the GptHDF5DataProcessor is used by multiple models, indicating that these models share similar data processing requirements.

The data processor is responsible for tasks such as data loading, tokenization, and formatting, which are necessary to prepare the data for training or inference. By using a data processor like GptHDF5DataProcessor, models can focus on their core functionality, while the data processor handles the data preparation tasks.

In summary, models interface with DataProcessors through a specific data processor associated with each model, which handles data preparation tasks, allowing the models to focus on their core functionality.

Exploring the Modelzoo

github.com/Cerebras/modelzoo

[ModelZoo docs overview](#)



Custom Training Loops with the cstorcht API

1. Import cstorcht package

Import

```
1 import torch
2 import cerebras_pytorch as cstorcht
3
4 model: torch.nn.Module = Model()
5 model = cstorcht.compile(model, "CSX")
6 loss_fn = torch.nn.NLLLoss()
7 optimizer = cstorcht.optim.SGD(model.parameters(), lr=0.01)
8 dataloader = cstorcht.utils.data.DataLoader(get_train_dataloader)
9 executor = cstorcht.utils.data.DataExecutor(
10     dataloader
11 )
12
13 @cstorcht.trace
14 def training_step(inputs, targets):
15     optimizer.zero_grad()
16     outputs = model(inputs)
17     loss = loss_fn(outputs, targets)
18     loss.backward()
19     optimizer.step()
20     return loss
21
22 @cstorcht.step_closure
23 def print_loss(loss: torch.Tensor):
24     print(f"Loss: {loss.item()}")
25
26 for inputs, targets in executor:
27     loss = training_step(inputs, targets)
28     print_loss(loss)
```

Custom Training Loops with the cstorches API

1. Import cstorches package
2. Define the model
 - Our model is defined as if running on a single device
 - We use the familiar PyTorch API with some drop-in replacements from cstorches
 - The dataloader gets wrapped in a cstorches data executor

Define
Model

```
1  import torch
2  import cerebras_pytorch as cstorches
3
4  model: torch.nn.Module = Model()
5  model = cstorches.compile(model, "CSX")
6  loss_fn = torch.nn.NLLLoss()
7  optimizer = cstorches.optim.SGD(model.parameters(), lr=0.01)
8  dataloader = cstorches.utils.data.DataLoader(get_train_dataloader)
9  executor = cstorches.utils.data.DataExecutor(
10     dataloader
11 )
12
13 @cstorches.trace
14 def training_step(inputs, targets):
15     optimizer.zero_grad()
16     outputs = model(inputs)
17     loss = loss_fn(outputs, targets)
18     loss.backward()
19     optimizer.step()
20     return loss
21
22 @cstorches.step_closure
23 def print_loss(loss: torch.Tensor):
24     print(f"Loss: {loss.item()}")
25
26 for inputs, targets in executor:
27     loss = training_step(inputs, targets)
28     print_loss(loss)
```


Custom Training Loops with the cstorcht API

1. Import cstorcht package
2. Define the model
 - Our model is defined as if running on a single device
 - We use the familiar PyTorch API with some drop-in replacements from cstorcht
 - The dataloader gets wrapped in a cstorcht data executor
3. Create the training loop method
 - Nothing novel here, except the decorator
 - `cstorcht.trace()` makes sure to capture the computation graph used in each training step

Define
Training
Loop

```
1  import torch
2  import cerebras_pytorch as cstorcht
3
4  model: torch.nn.Module = Model()
5  model = cstorcht.compile(model, "CSX")
6  loss_fn = torch.nn.NLLLoss()
7  optimizer = cstorcht.optim.SGD(model.parameters(), lr=0.01)
8  dataloader = cstorcht.utils.data.DataLoader(get_train_dataloader)
9  executor = cstorcht.utils.data.DataExecutor(
10     dataloader
11 )
12
13 @cstorcht.trace
14 def training_step(inputs, targets):
15     optimizer.zero_grad()
16     outputs = model(inputs)
17     loss = loss_fn(outputs, targets)
18     loss.backward()
19     optimizer.step()
20     return loss
21
22 @cstorcht.step_closure
23 def print_loss(loss: torch.Tensor):
24     print(f"Loss: {loss.item()}")
25
26 for inputs, targets in executor:
27     loss = training_step(inputs, targets)
28     print_loss(loss)
```

Custom Training Loops with the cstorcht API

1. Import cstorcht package
2. Define the model
 - Our model is defined as if running on a single device
 - We use the familiar PyTorch API with some drop-in replacements from cstorcht
 - The dataloader gets wrapped in a cstorcht data executor
3. Create the training loop method
 - Nothing novel here, except the decorator
 - `cstorcht.trace()` makes sure to capture the computation graph used in each training step
4. Run the training loop
 - Compiles the model during the first step and then starts asynchronous execution
 - Outputs (losses) are retrieved from the wafer as they become available

Train

```
1  import torch
2  import cerebras_pytorch as cstorcht
3
4  model: torch.nn.Module = Model()
5  model = cstorcht.compile(model, "CSX")
6  loss_fn = torch.nn.NLLLoss()
7  optimizer = cstorcht.optim.SGD(model.parameters(), lr=0.01)
8  dataloader = cstorcht.utils.data.DataLoader(get_train_dataloader)
9  executor = cstorcht.utils.data.DataExecutor(
10     dataloader
11 )
12
13 @cstorcht.trace
14 def training_step(inputs, targets):
15     optimizer.zero_grad()
16     outputs = model(inputs)
17     loss = loss_fn(outputs, targets)
18     loss.backward()
19     optimizer.step()
20     return loss
21
22 @cstorcht.step_closure
23 def print_loss(loss: torch.Tensor):
24     print(f"Loss: {loss.item()}")
25
26 for inputs, targets in executor:
27     loss = training_step(inputs, targets)
28     print_loss(loss)
```


Custom Training Loops with the cstorches API

- Scale out to multiple CS-3s with a one-argument configuration change
- Near-linear scaling happens automatically
- We don't need to change...
 - Model code
 - Training loop
 - Effective batch size
- It's easy to supply the value programmatically for edit-free scaling

Scale out

```
1  import torch
2  import cerebras_pytorch as cstorches
3
4  model: torch.nn.Module = Model()
5  model = cstorches.compile(model, "CSX")
6  loss_fn = torch.nn.NLLLoss()
7  optimizer = cstorches.optim.SGD(model.parameters(), lr=0.01)
8  dataloader = cstorches.utils.data.DataLoader(get_train_dataloader)
9  executor = cstorches.utils.data.DataExecutor(
10     dataloader, cs_config=cstorches.utils.CSConfig(num_csx=16)
11 )
12
13 @cstorches.trace
14 def training_step(inputs, targets):
15     optimizer.zero_grad()
16     outputs = model(inputs)
17     loss = loss_fn(outputs, targets)
18     loss.backward()
19     optimizer.step()
20     return loss
21
22 @cstorches.step_closure
23 def print_loss(loss: torch.Tensor):
24     print(f"Loss: {loss.item()}")
25
26 for inputs, targets in executor:
27     loss = training_step(inputs, targets)
28     print_loss(loss)
```



Model Training with Cerebras

Scaling Up and Out

Training an 8B model on a single CS-3

Define the model

- Develop in PyTorch for a single device
- Parameterize in a yaml file
 - Model width and depth
 - Number of training steps
 - Data processors to use
 - Number of Cerebras devices to use

Train the model

- Point to the model parameters
- Optional: override number of devices
- Run!

params_llama3_8b.yaml

```
### LLaMA-3 8B

hidden_size: 4096
num_hidden_layers: 32
num_heads: 32
```

training:

```
cszoo fit params_llama3_8b.yaml \
--num_csx 1
```

Scaling from 8B model to 70B

Define the model

- Same code as for Llama-3 8B
- Different yaml parameterization
- 70B model code is still for a *single* device

params_llama3_70b.yaml

```
### LLaMA-3 70B

hidden_size: 8192
num_hidden_layers: 80
num_heads: 64
```

Train the model

- Point to the upscaled model parameters
- The rest is the same as with Llama-3 8B
- Run!

training:

```
cszoo fit params_llama3_70b.yaml \
--num_csx 1
```

Scaling to 1T

Define the model

- Same code
- Different yaml parameterization
- **1T model code is still for a *single* device**

Train the model

- Point to the upscaled model parameters
- The rest is the same as with Llama-3 8B
- Run!

params_llama3_1010b.yaml

```
### LLaMA-3 1010B  
  
hidden_size: 20480  
num_hidden_layers: 200  
num_heads: 160
```

Parameter count: 1,007,662,223,360

training:

```
cszoo fit params_llama3_1010b.yaml \  
--num_csx 1
```

Scaling up to **24xCS-3**

Define the model

- Same code
- Same yaml file

params_llama3_70b.yaml

```
### LLaMA-3 70B

hidden_size: 8192
num_hidden_layers: 80
num_heads: 64
```

Train the model

- Indicate **24** systems in the command line
- Nothing else changes!
- Run!

training:

```
cszoo fit params_llama3_70b.yaml \  
--num_csx 24
```

Throughput Comparison

- 1xCS-3

Preparing to execute using 1 CSX

Train Device=CSX, Step=10, Loss=37.29254, Rate=0.69 samples/sec, GlobalRate=0.68 samples/sec

- 24xCS-3

Preparing to execute using 24 CSX

Train Device=CSX, Step=12, Loss=1.92508, Rate=16.32 samples/sec, GlobalRate=16.24 samples/sec

Linear scaling: $16.24 / 0.68 \approx 23.88x$ Speedup (99.5% scaling)*

*Model executed on 24xCSX systems had 2x larger vocabulary than the model executed on 1xCSX.
Running the exact same model would result in 100% scaling.

Monitoring results with TensorBoard

1. Activate Python environment (if not already activated)

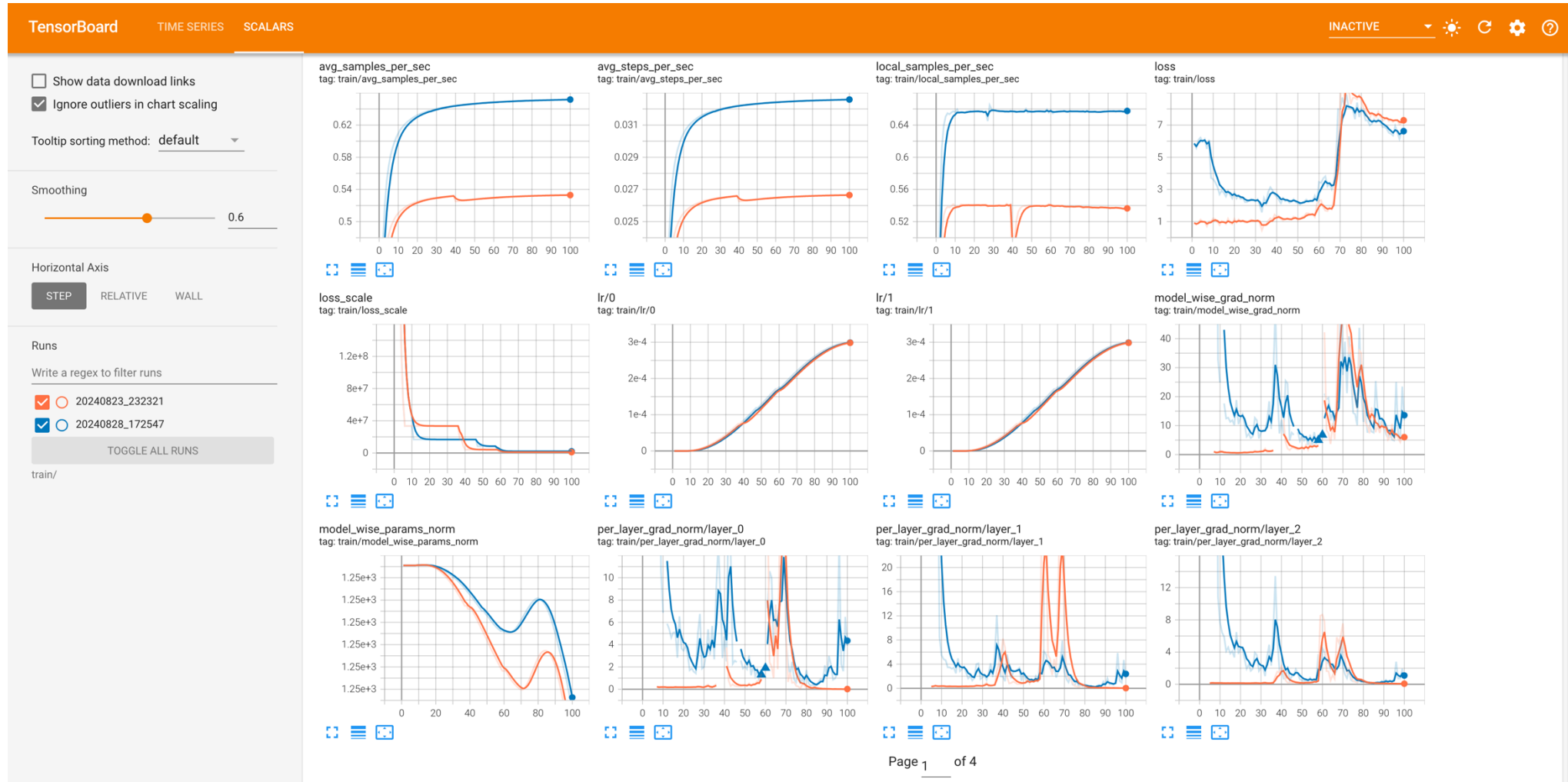
```
$ source venvs/2.3.1/bin/activate
```

2. Launch TensorBoard choosing the model directory of the run

```
$ tensorboard --logdir_spec={your_model_dir}/ --bind_all --port=6006
```

3. Open the provided link in the output from your local machine, port forwarding as needed

Monitoring results with TensorBoard



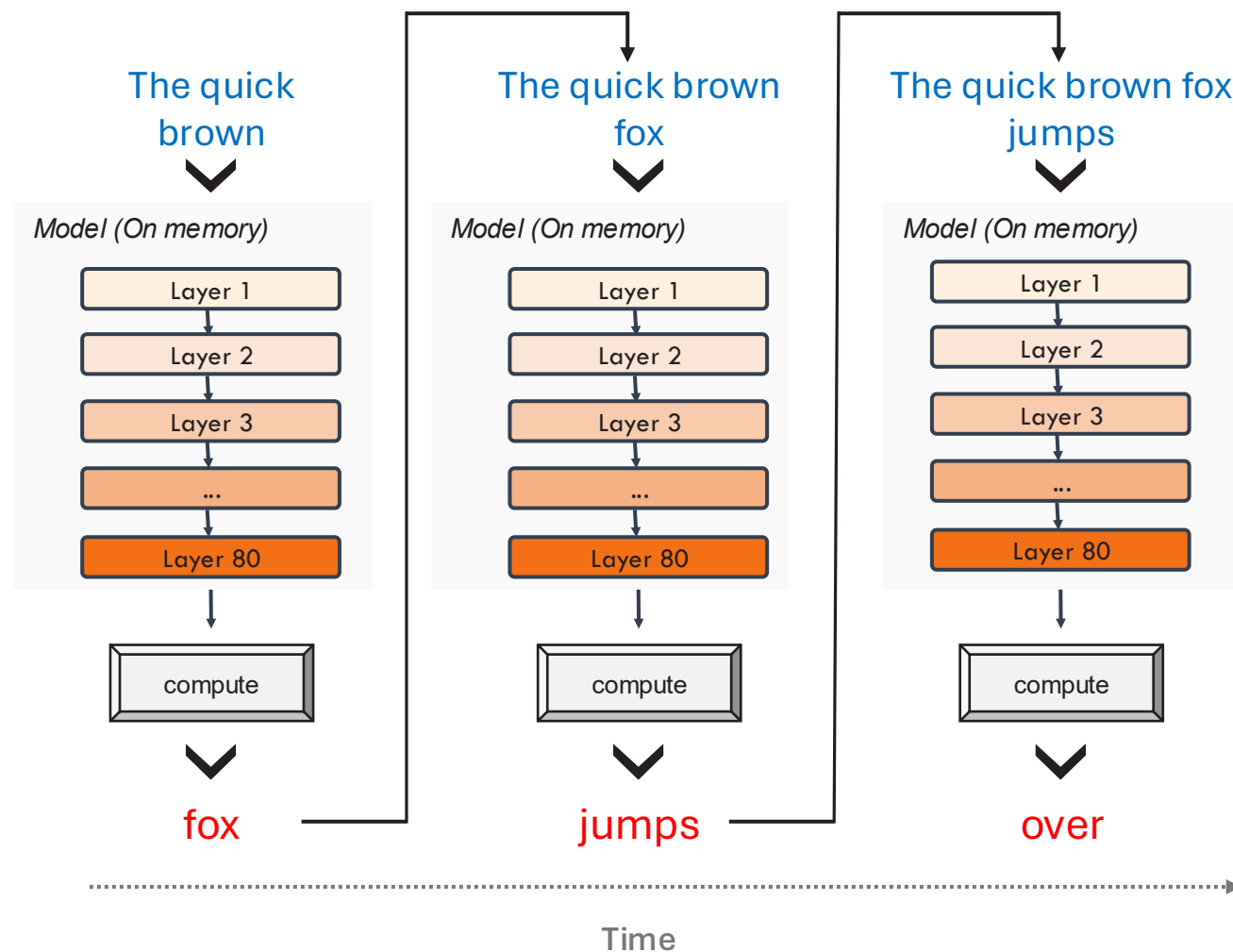


Cerebras Inference

Powering Instant AI

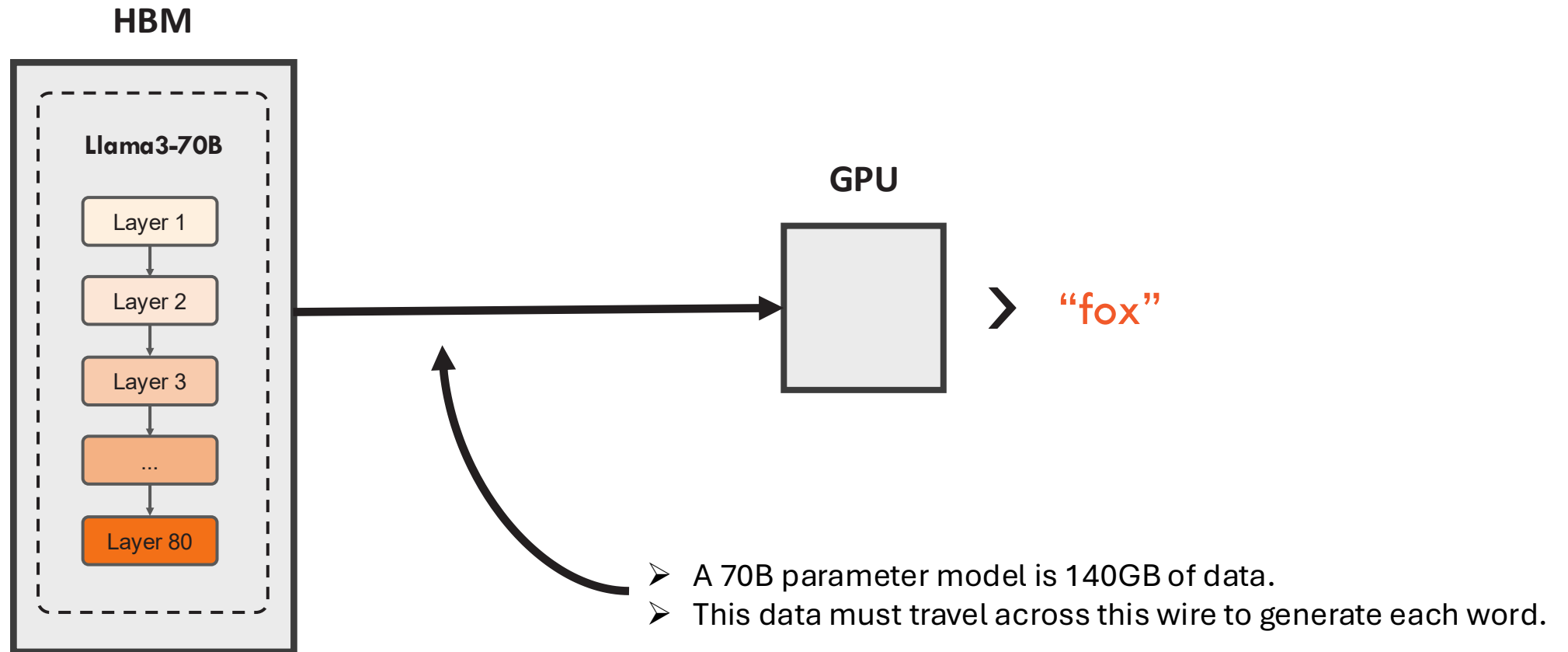
GenAI inference is a **memory bandwidth** problem

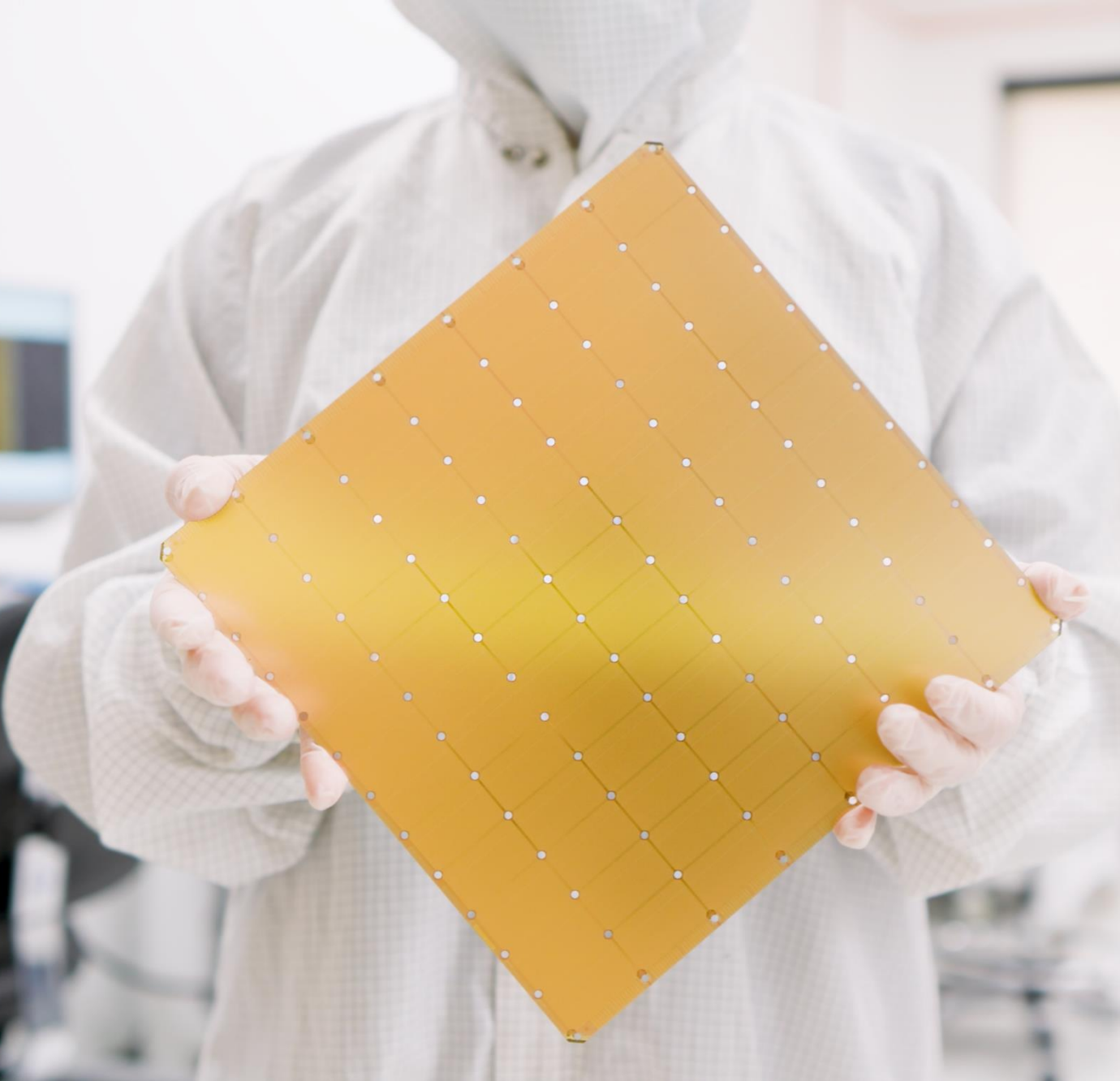
- Generating 1000 tokens takes 1000 serial passes through the model
- Each pass requires reading all model parameters from memory
- Low memory bandwidth is the bottleneck for generation performance



GPUs are limited by memory bandwidth

In a GPU, most of the memory is off-chip, far away from the compute.





Big chip = huge memory bandwidth

4 trillion transistors

46,225 mm² silicon

900,000 AI cores

125 Petaflops of AI compute

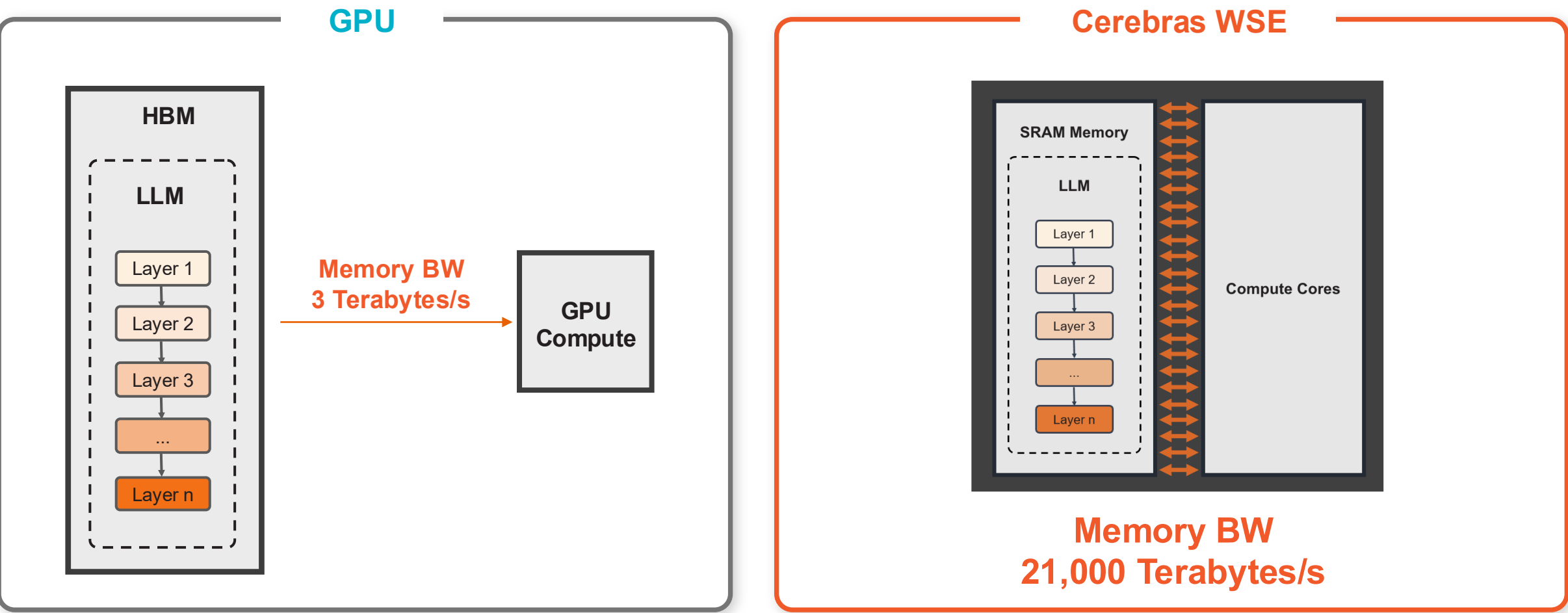
44 Gigabytes of on-chip memory

21 PByte/s memory bandwidth

214 Pbit/s fabric bandwidth

5nm TSMC process

Cerebras WSE delivers 7,000x more memory bandwidth than Nvidia H100 Resulting in >70x faster inference speeds

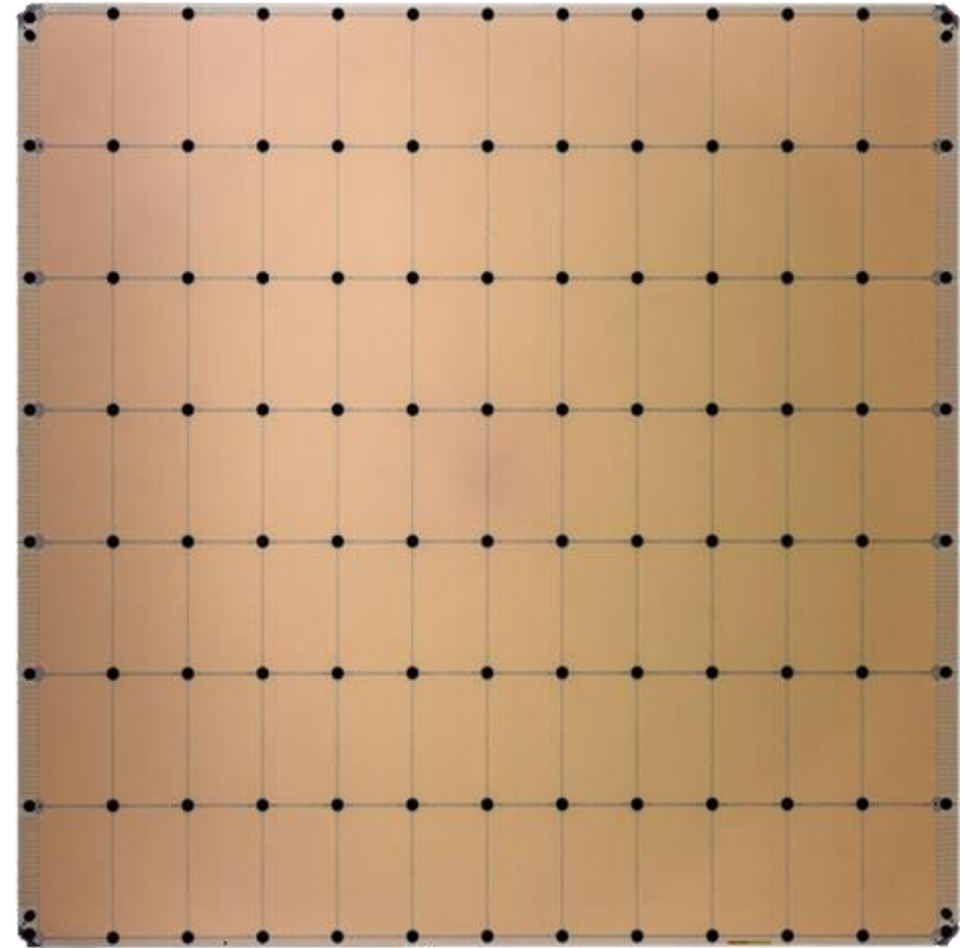
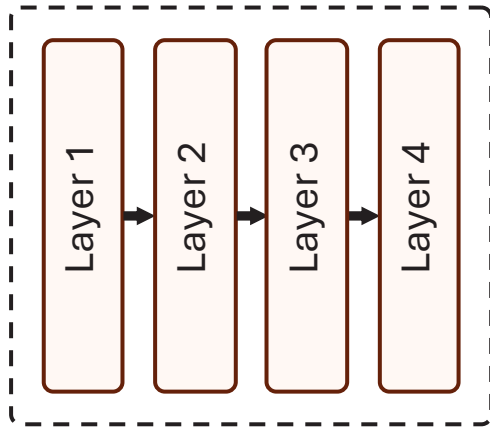


The WSE puts massive amounts of high-performance SRAM directly on the wafer
Faster memory, closer to compute, delivers faster results

Pipeline execution on a single chip

Massive memory bandwidth enables opposite execution model

- GPU: multiple chips to run a single layer
- Cerebras: fraction of a chip to run a single layer



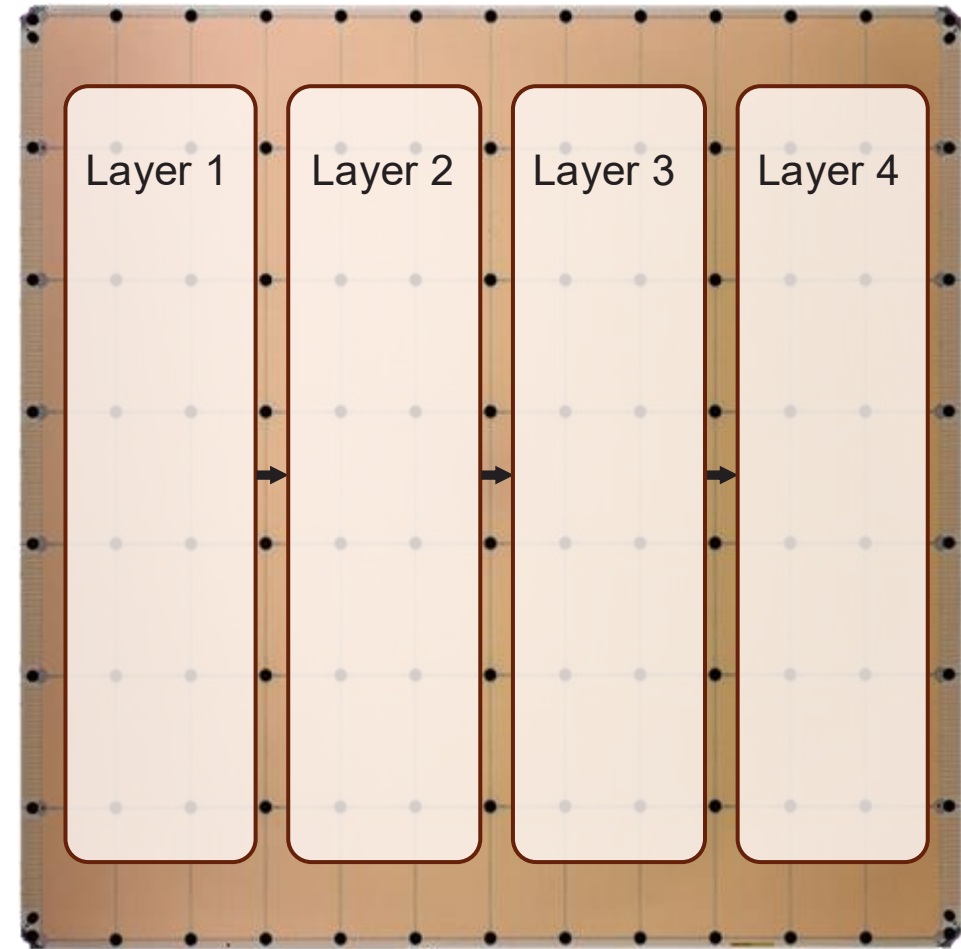
Pipeline execution on a single chip

Massive memory bandwidth enables opposite execution model

- GPU: multiple chips to run a single layer
- Cerebras: fraction of a chip to run a single layer

Pipeline mapping of model

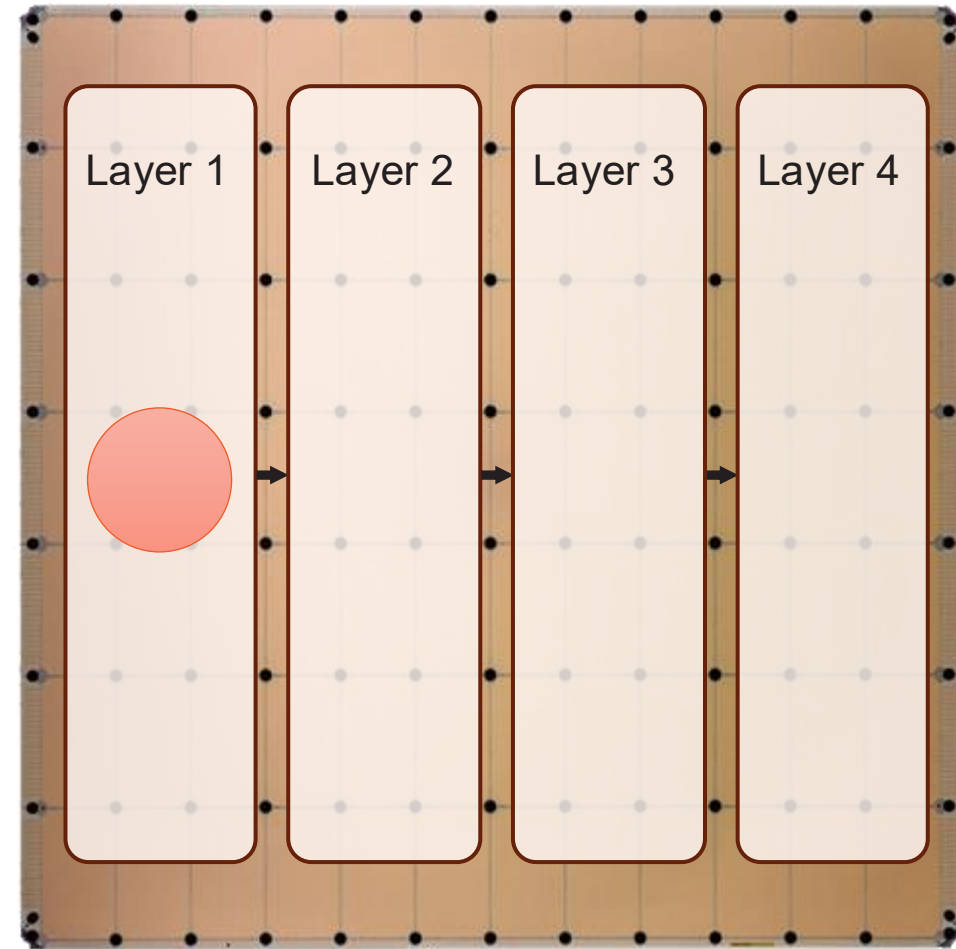
- Model layers are mapped to wafer regions
- Size of wafer region determined based on memory and compute requirements
- Model weights and KV cache stored locally in the region memory close to the compute



Pipeline execution

A low latency pipeline

- Each wafer region processes 1 token
 - Enough memory bandwidth to run local batch size 1
 - Memory to feed compute datapath at full speed
 - Enabling optimized performance for matrix*vector
 - Local fabric interconnect to maintain low latency

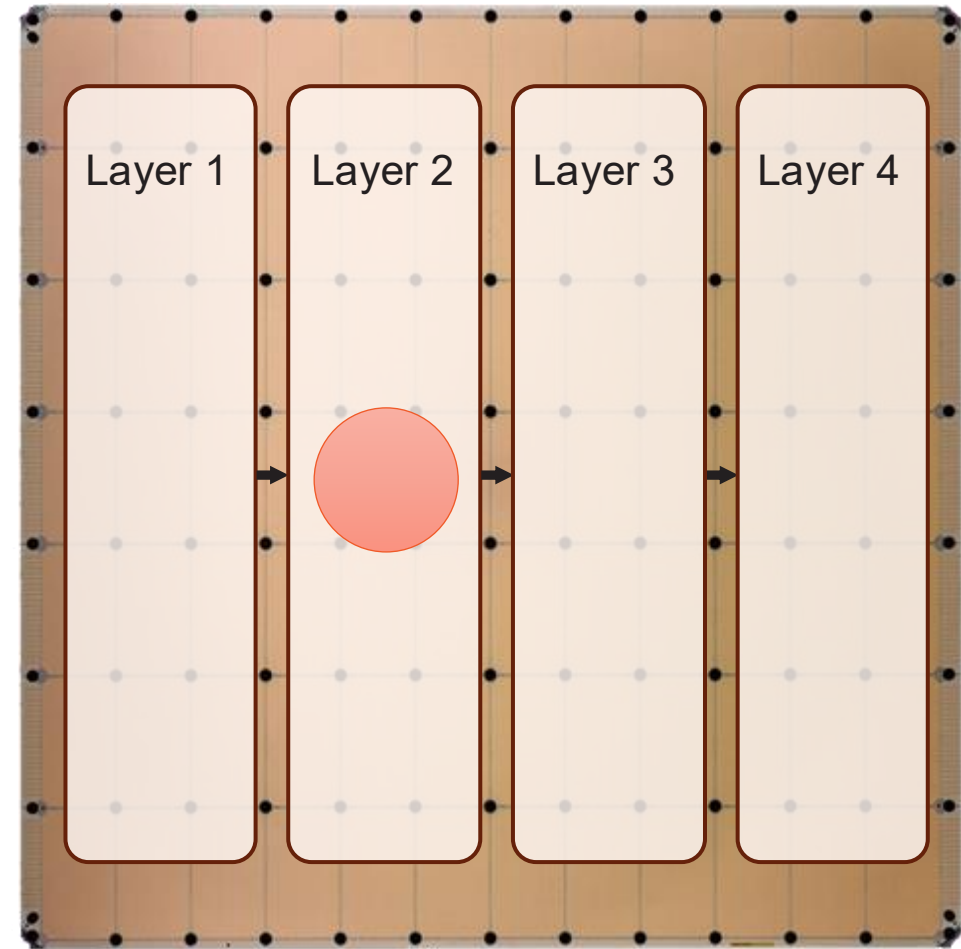


Token

Pipeline execution

A low latency pipeline

- Each wafer region processes 1 token
 - Enough memory bandwidth to run local batch size 1
 - Memory to feed compute datapath at full speed
 - Enabling optimized performance for matrix*vector
 - Local fabric interconnect to maintain low latency
- Regions are placed to reduce latency
 - The next region is physically adjacent
 - Virtually no latency between pipe stages
 - Possible because it's all done on-chip

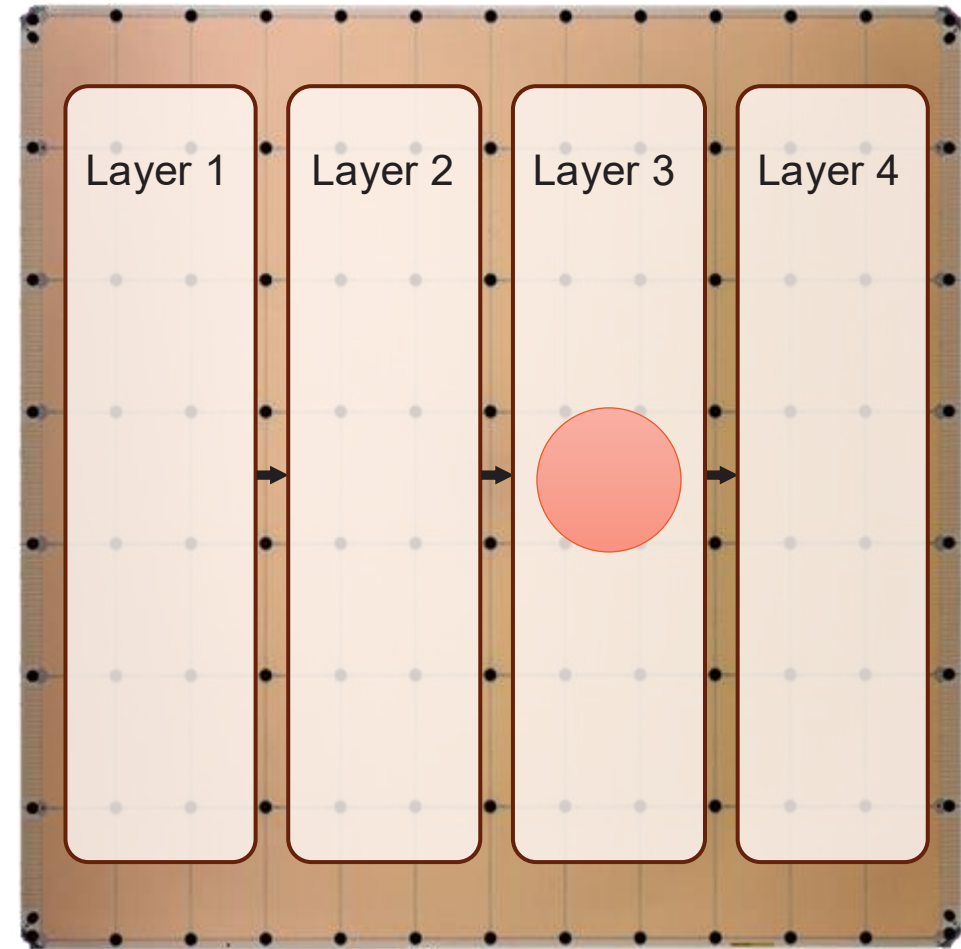


Token

Pipeline execution

A low latency pipeline

- Each wafer region processes 1 token
 - Enough memory bandwidth to run local batch size 1
 - Memory to feed compute datapath at full speed
 - Enabling optimized performance for matrix*vector
 - Local fabric interconnect to maintain low latency
- Regions are placed to reduce latency
 - The next region is physically adjacent
 - Virtually no latency between pipe stages
 - Possible because it's all done on-chip

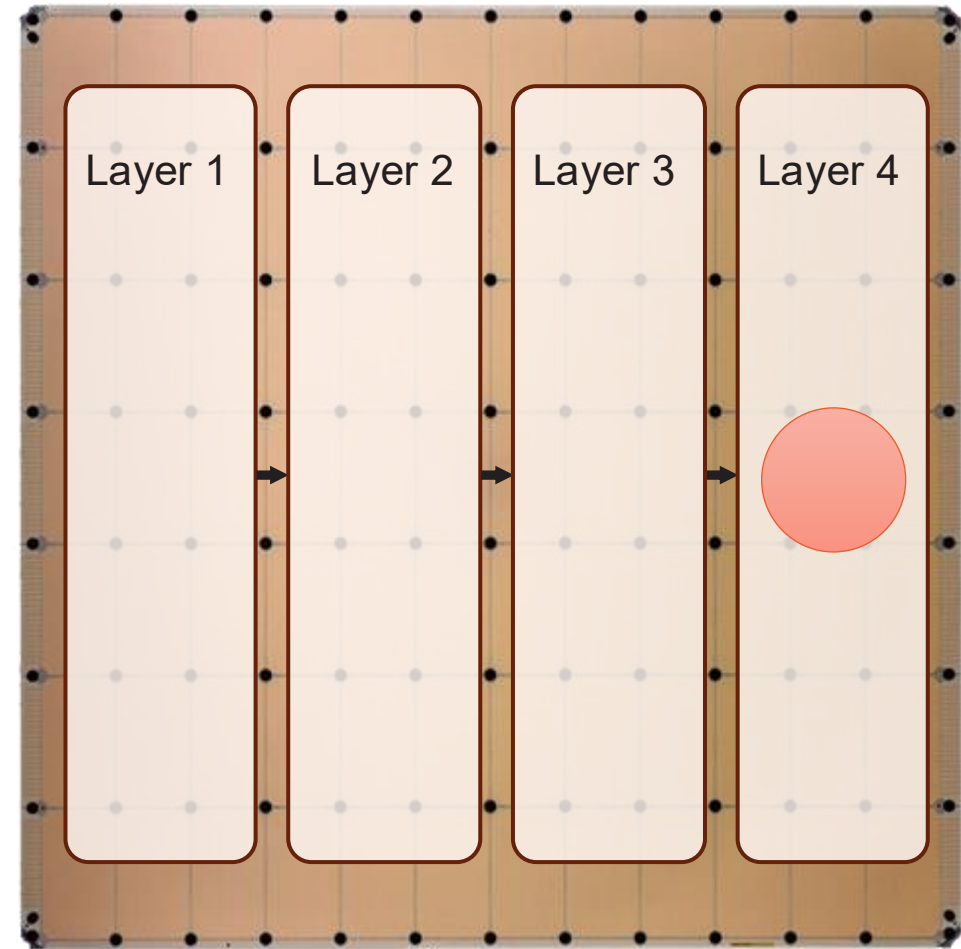


Token

Pipeline execution

A low latency pipeline

- Each wafer region processes 1 token
 - Enough memory bandwidth to run local batch size 1
 - Memory to feed compute datapath at full speed
 - Enabling optimized performance for matrix*vector
 - Local fabric interconnect to maintain low latency
- Regions are placed to reduce latency
 - The next region is physically adjacent
 - Virtually no latency between pipe stages
 - Possible because it's all done on-chip



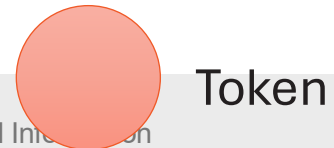
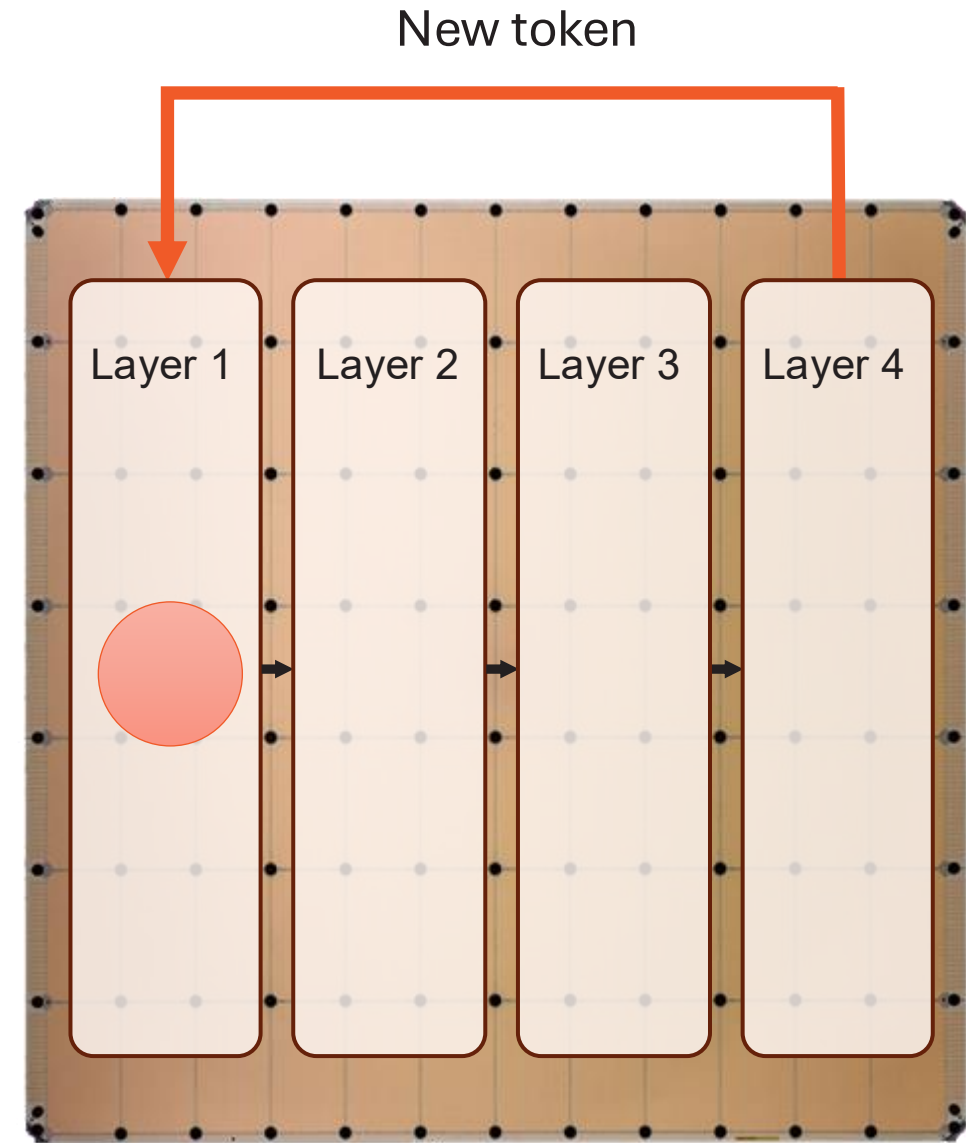
Token

Pipeline execution

A low latency pipeline

- Each wafer region processes 1 token
 - Enough memory bandwidth to run local batch size 1
 - Memory to feed compute datapath at full speed
 - Enabling optimized performance for matrix*vector
 - Local fabric interconnect to maintain low latency
- Regions are placed to reduce latency
 - The next region is physically adjacent
 - Virtually no latency between pipe stages
 - Possible because it's all done on-chip
- Last region output cycled back to generate next token

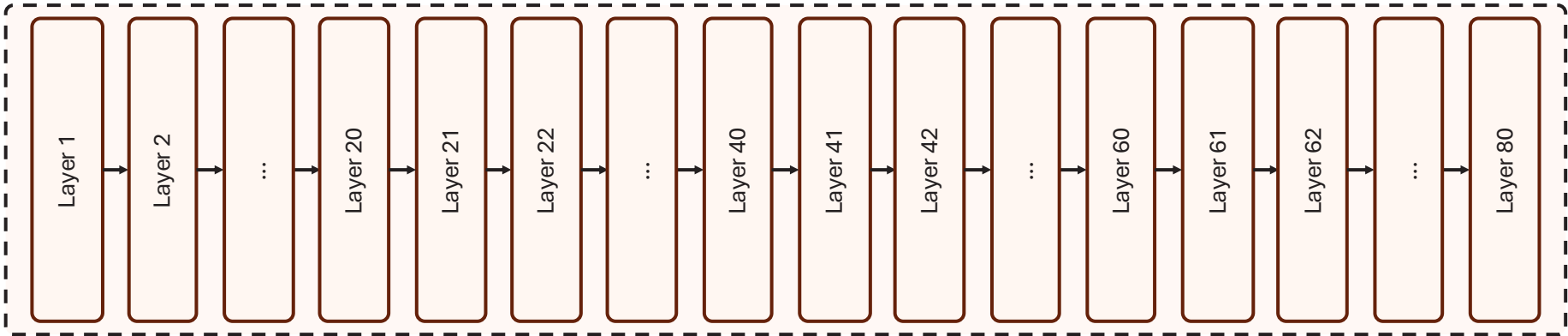
Pipelined execution enables super-fast token generation



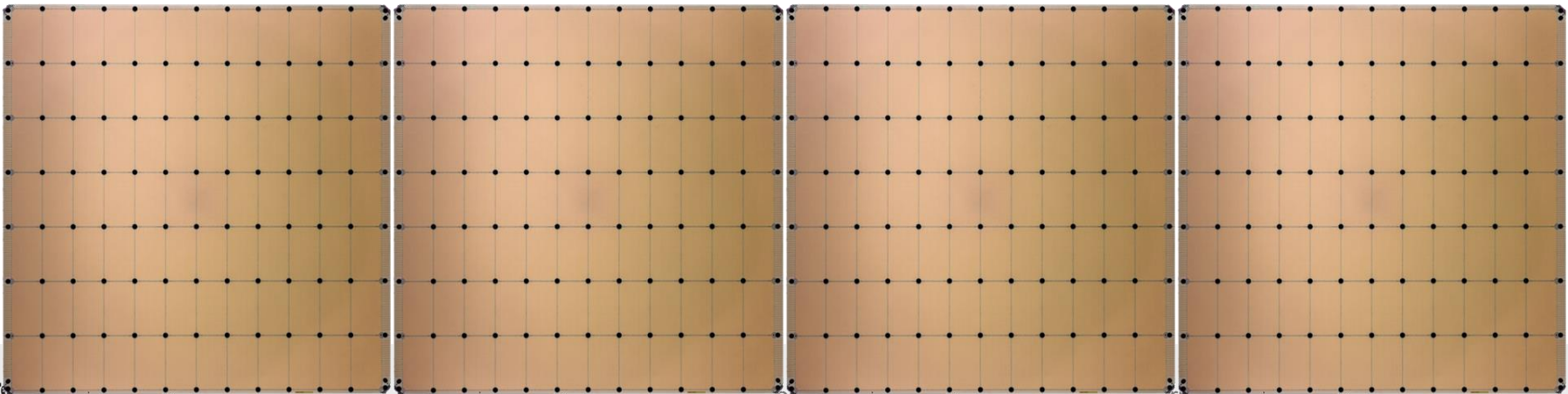
The background is a dark gray with a complex, abstract pattern of concentric circles and radial lines, resembling a stylized spiral or a technical diagram. The lines are in various shades of gray, creating a sense of depth and movement.

Multiple Systems

Mapping to multiple wafers



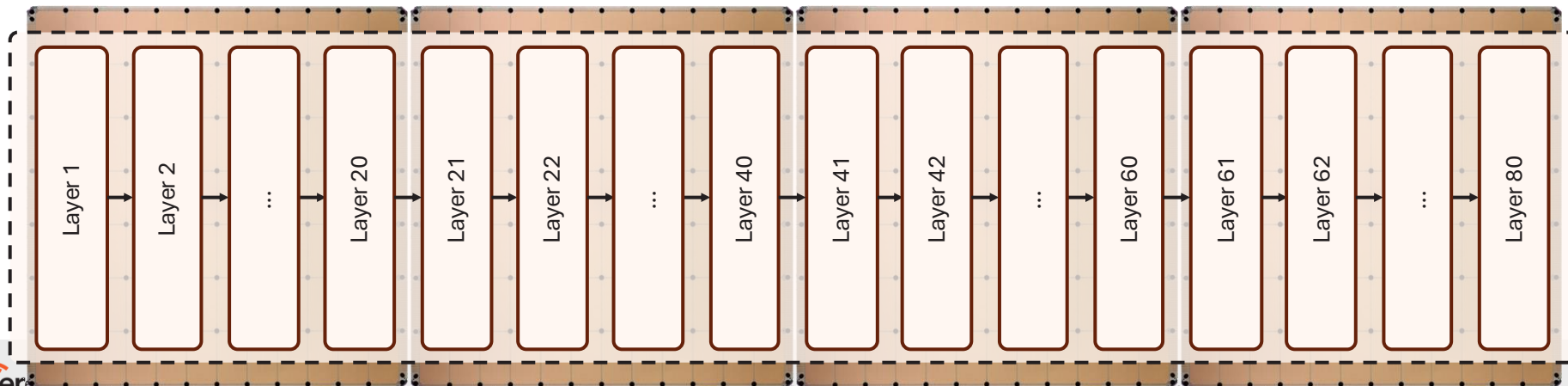
Llama3.1-70B
70 billion parameters
140 GB of memory (FP16)



4x WSE-3
176GB of SRAM

Mapping to multiple wafers

- Models that require more memory are mapped to multiple wafers
- Keep almost all high communication on wafer, using internal high bandwidth fabric
- Transfer only activations between wafers, requiring relatively lower bandwidth
- Using CS-3 low latency RDMA over Ethernet interconnect between systems



Llama3.1-70B

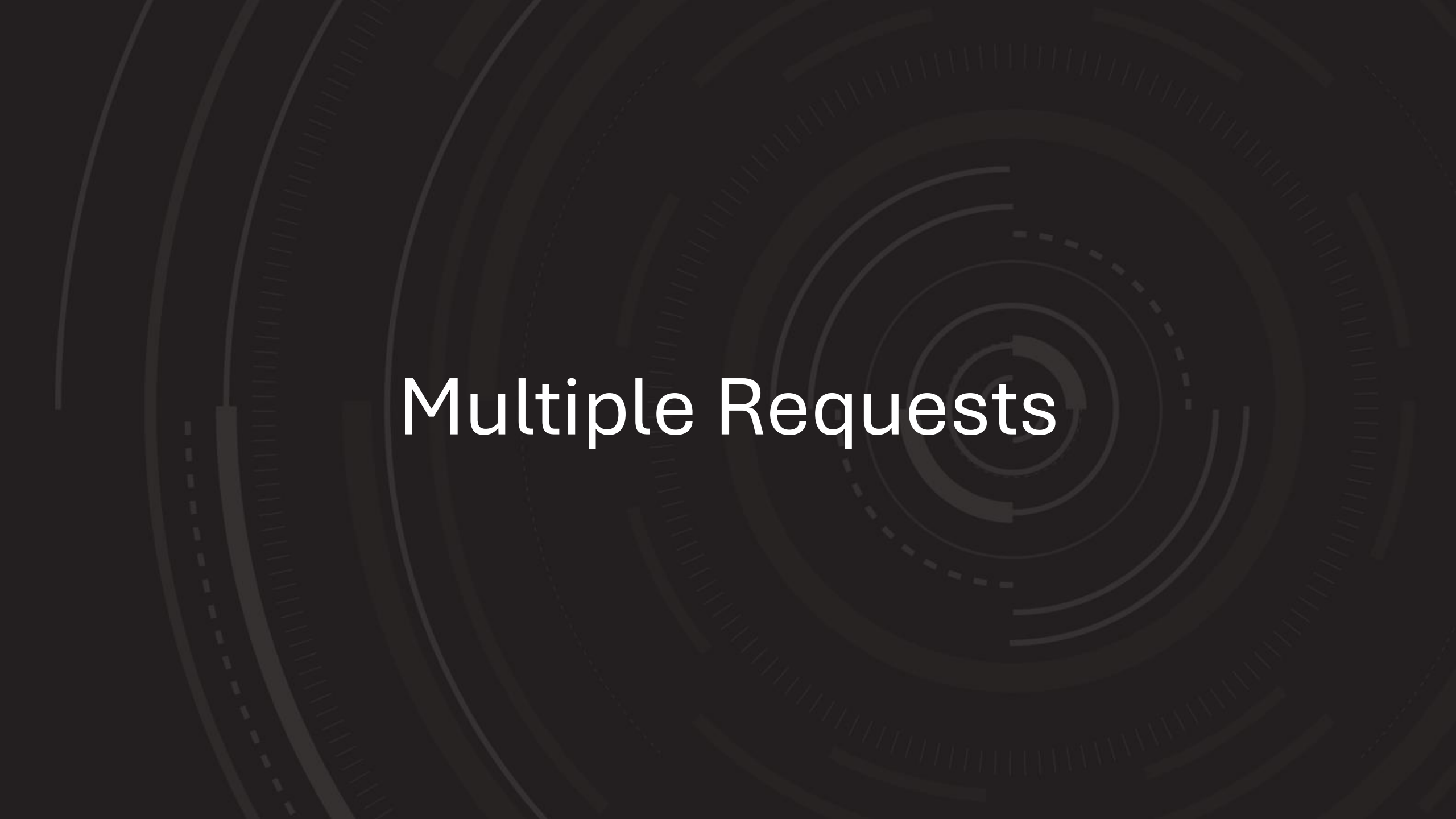
70 billion parameters

140 GB of memory (FP16)

4x WSE-3

176GB of SRAM

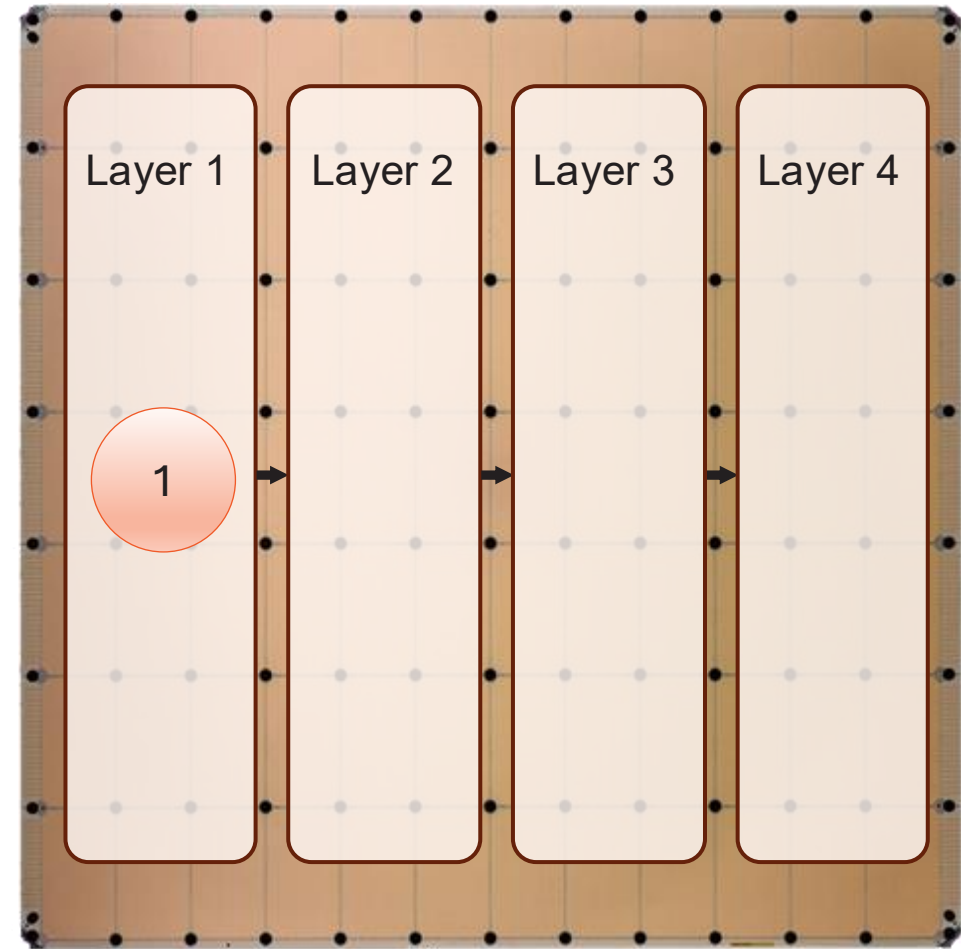
Multiple Requests

The background of the slide is a dark gray with a complex, abstract pattern. It features numerous concentric circles and radial lines, some of which are dashed, creating a sense of depth and movement. The pattern is centered around the text, which is a large, white, sans-serif font.

Multiple requests

More than enough memory bandwidth for single user enables high multi-user throughput

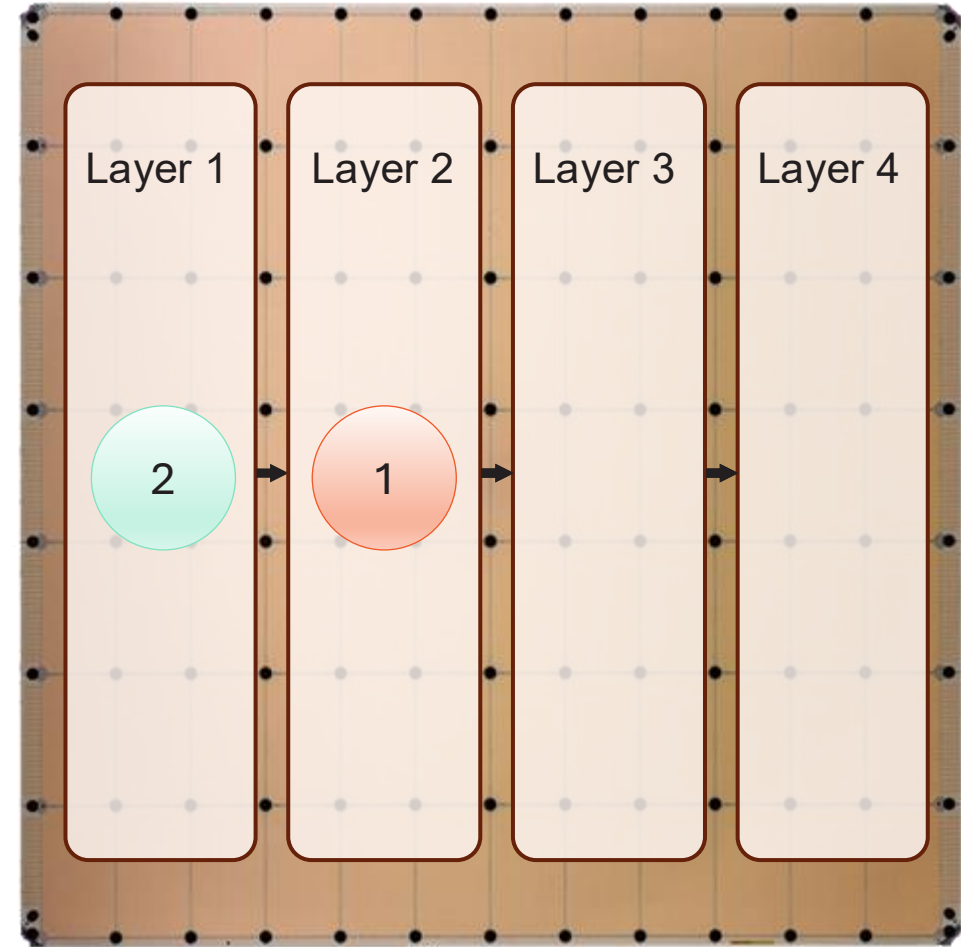
- Single user is using only a fraction of the total memory bandwidth
- We can use the extra bandwidth to support multiple users



Multiple requests

More than enough memory bandwidth for single user enables high multi-user throughput

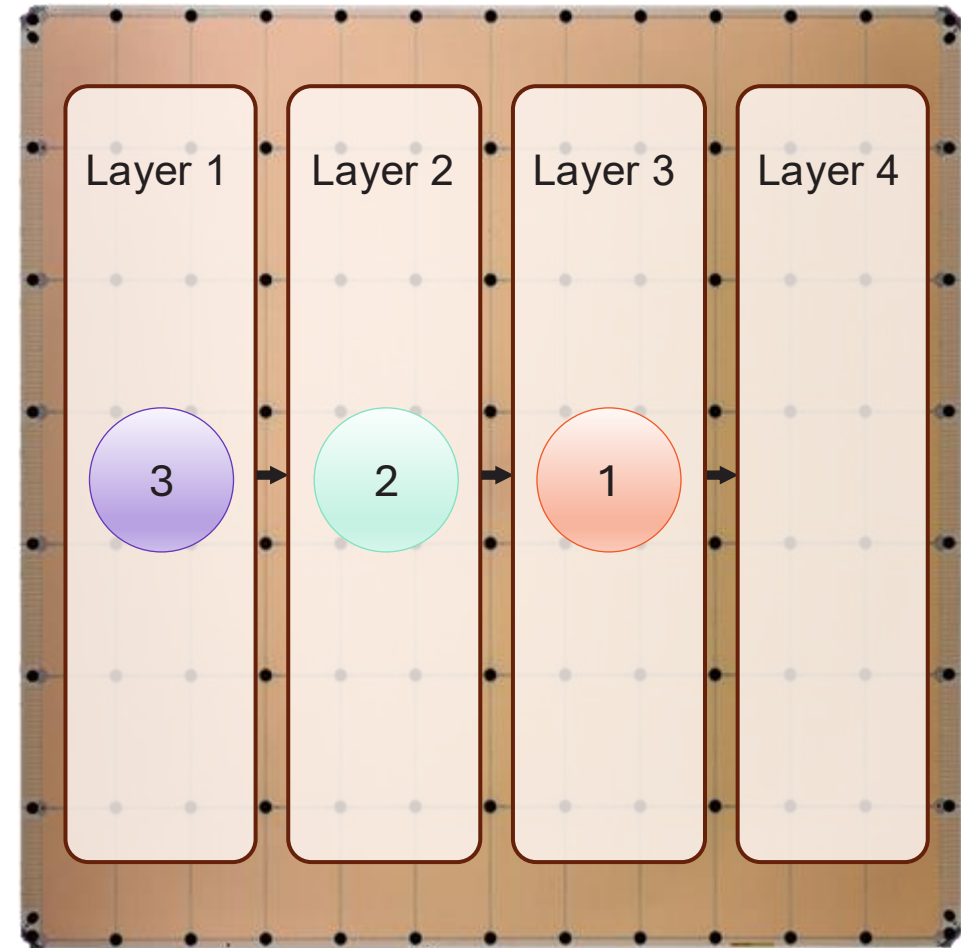
- Single user is using only a fraction of the total memory bandwidth
- We can use the extra bandwidth to support multiple users
- Additional users all run in parallel
- Each accessing the model simultaneously



Multiple requests

More than enough memory bandwidth for single user enables high multi-user throughput

- Single user is using only a fraction of the total memory bandwidth
- We can use the extra bandwidth to support multiple users
- Additional users all run in parallel
- Each accessing the model simultaneously
- Every user gets full performance
- All pipe stages to run at the same time

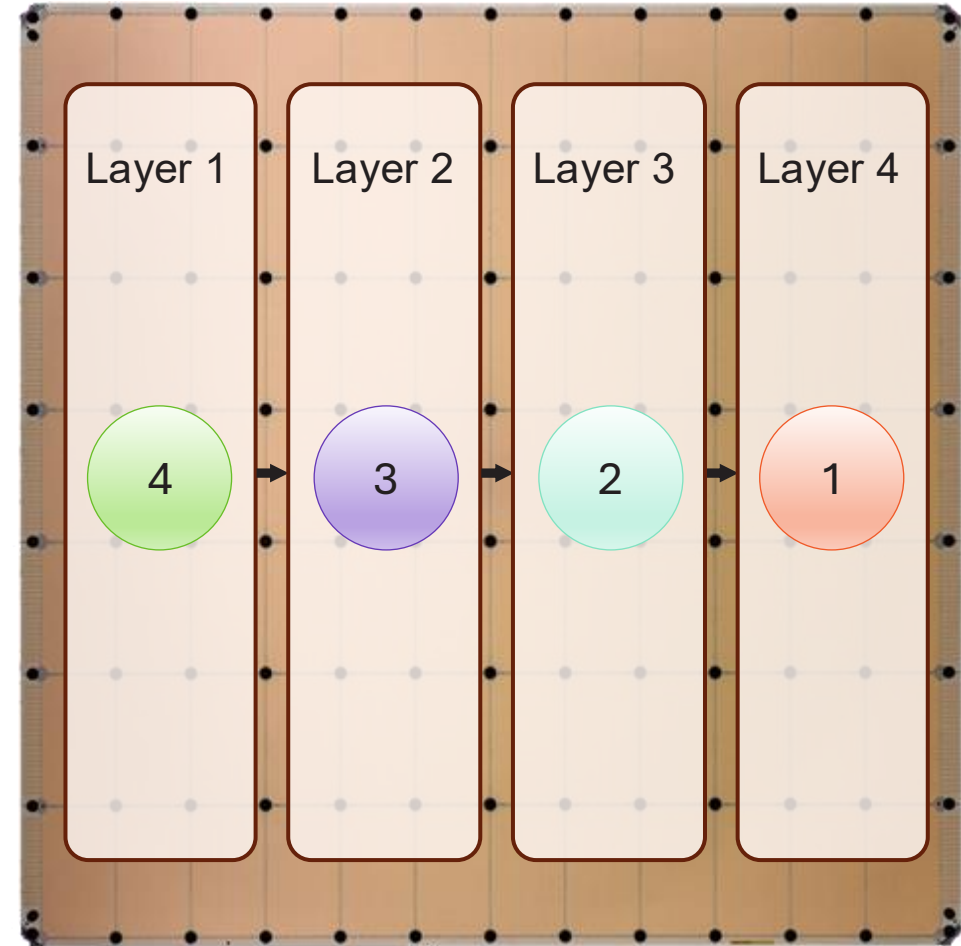


Multiple requests

More than enough memory bandwidth for single user enables high multi-user throughput

- Single user is using only a fraction of the total memory bandwidth
- We can use the extra bandwidth to support multiple users
- Additional users all run in parallel
- Each accessing the model simultaneously
- Every user gets full performance
- All pipe stages to run at the same time

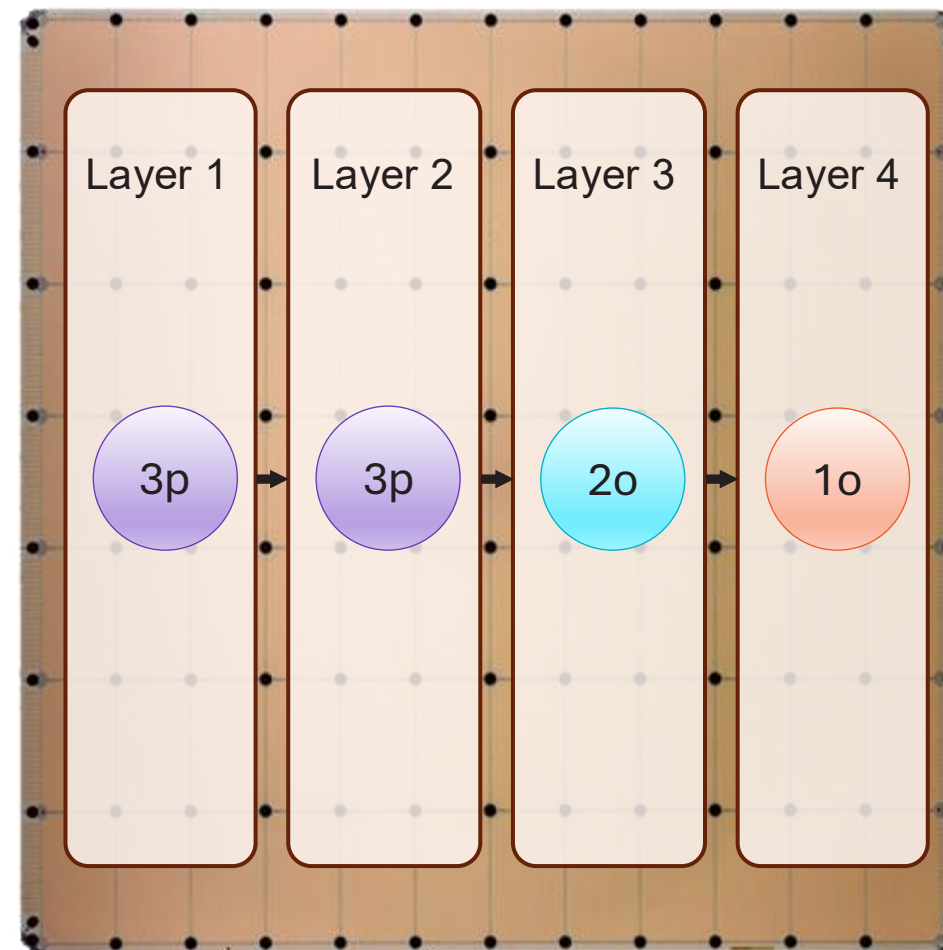
Full pipeline model parallelism on single chip



Continuous Batching

Fine-grained token scheduling enables natural continuous batching

- Token-by-token processing and pipeline execution gives native ability to do fine-grained mixing of streams, both prompt and outputs.
- Can dynamically interleave all sequences at fine granularity to give good balance of prompt and output performance simultaneously.
- As long as there is pipeline capacity, available sequences can be processed, no need to wait until sequence boundaries.
- Scheduler attempts to fill pipeline slots with whatever is available to drive up performance: prompt, output, draft





The Cerebras SDK

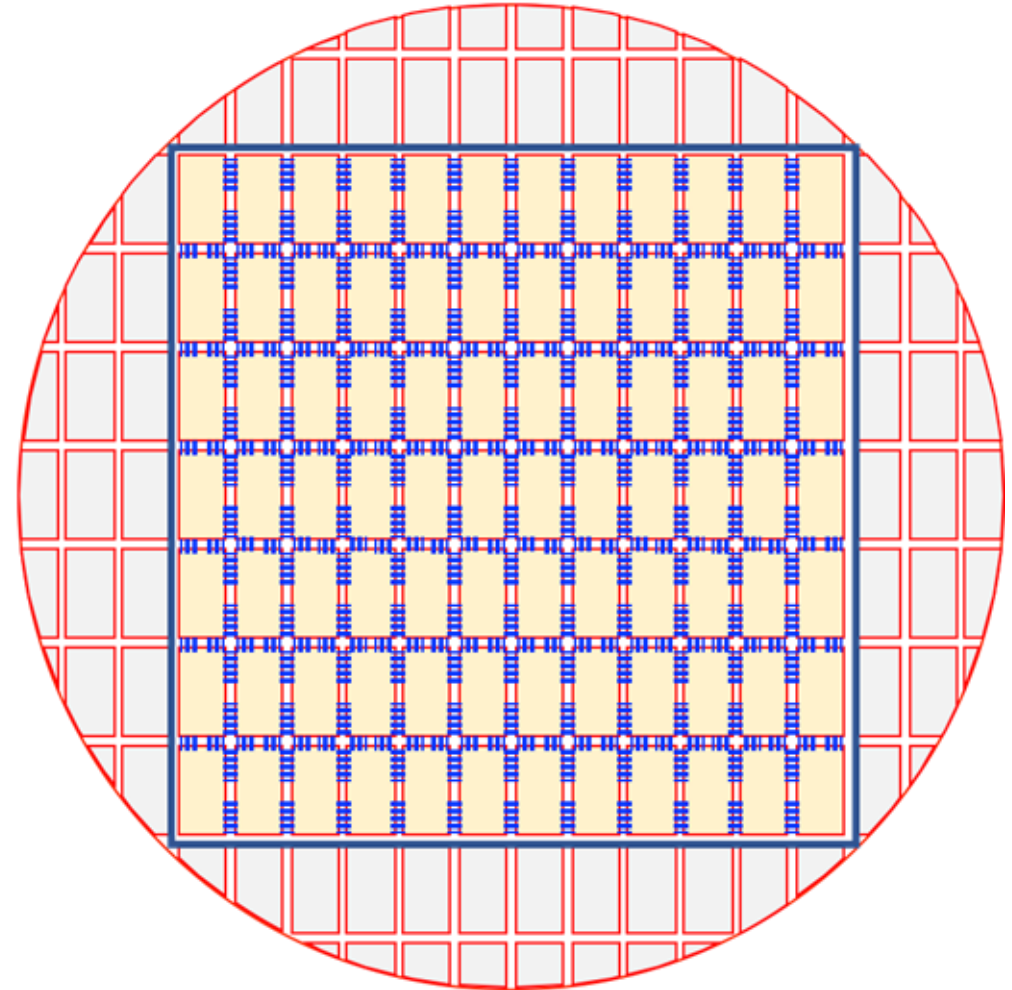
Empowering HPC Applications

Does your application scale poorly across nodes?

Examples: FFT-based solvers, particle simulators, non-linear problems with iterative solvers

The Cerebras solution:

- The WSE-3 has a fabric that is high bandwidth and low-latency, allowing for excellent parallel efficiency for non-linear and highly communicative codes
- The CS-3 system with WSE-3 has 900k cores and can fit problems on an individual chip that traditionally take 10s to hundreds of small compute nodes.
 - Each core is individually programmable



Is your application constrained by data access?

Examples: Stencil based PDE solvers, linear algebra solvers, signal processing, sparse tensor math, big data analysis

The Cerebras solution:

- The CS-3 system has 44 GB of SRAM uniformly distributed across the WSE-3 that is 1 cycle away from the processing element
 - Speeds up memory access by orders of magnitude
- The CS-3 system is capable of 1.2 Tb/s bandwidth onto the chip
 - Stream data onto the chip as required



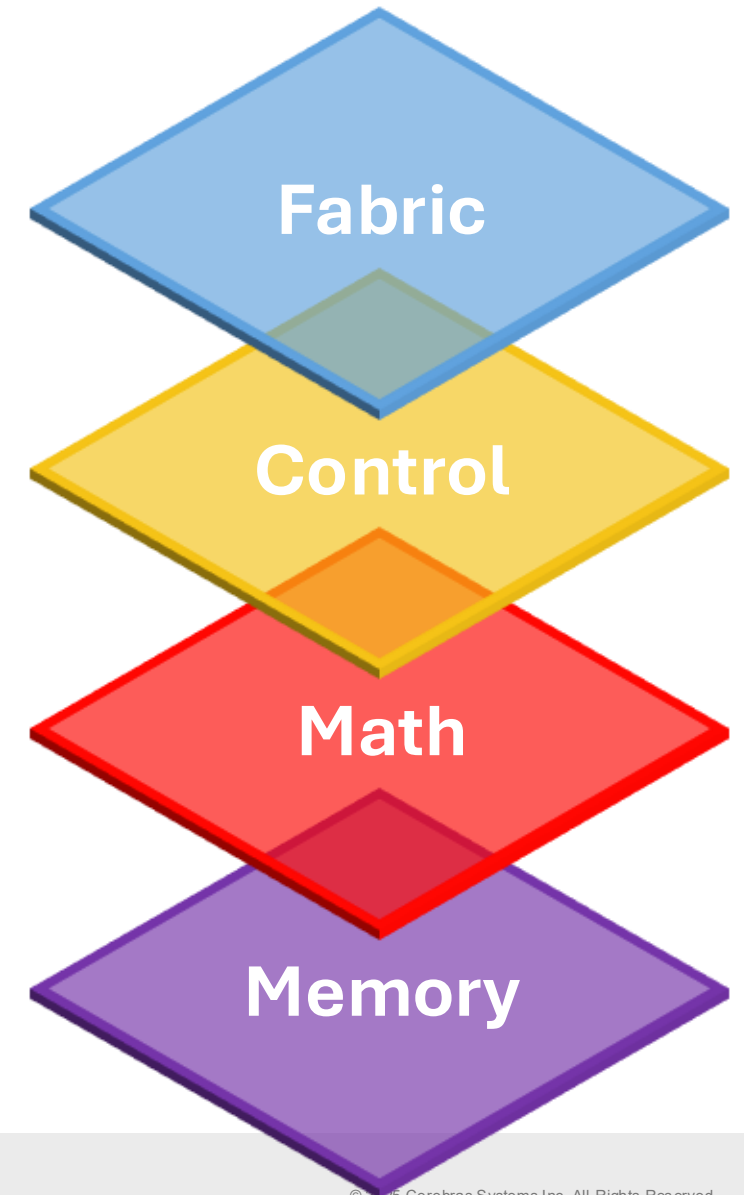
Completely Uniform Mesh

Numerical efficiency is all about bandwidth

Programmability for yet-to-exist algorithms

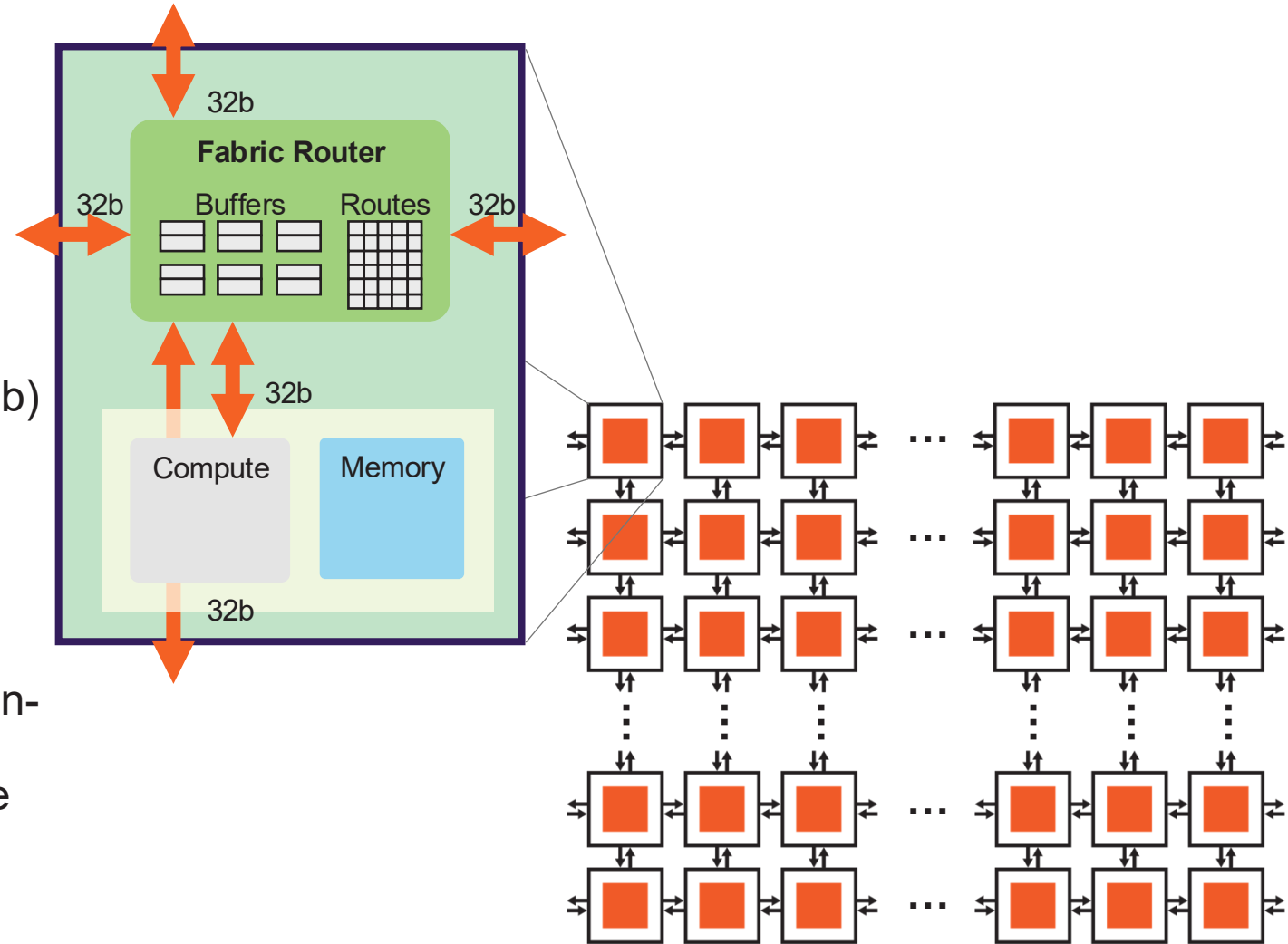
Numerical algorithms are the core of ML & HPC

Storage for activations and parameters



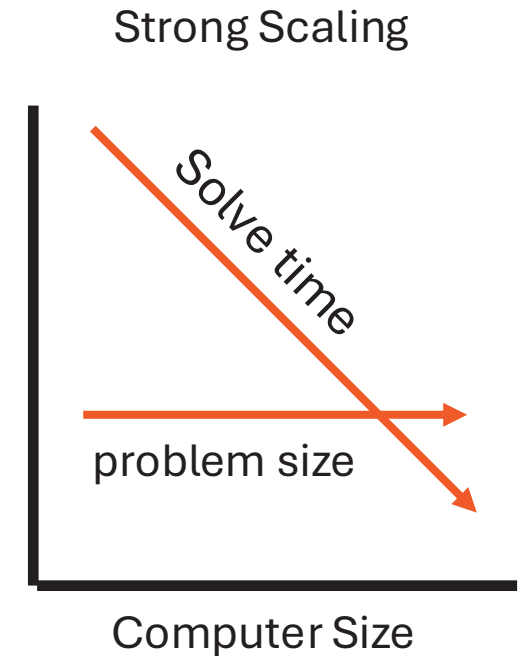
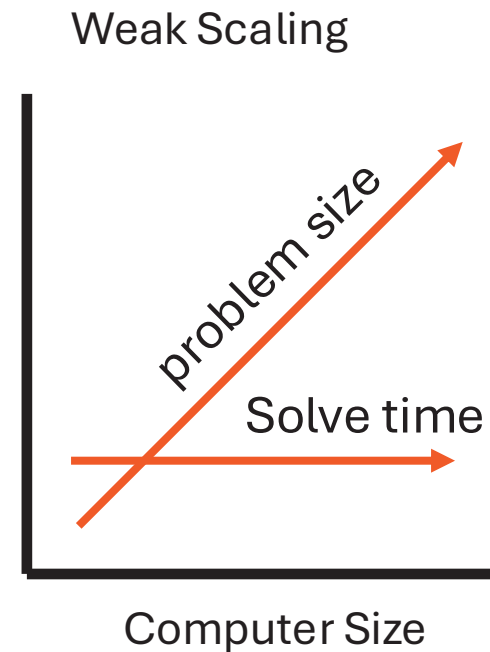
Efficient high performance from high bandwidth & low latency fabric

- 2D mesh topology with low overheads
- 5-port router to 4 neighbors and core
- 32b/cycle bidirectional data transfer
 - Individual packages are 32b
 - Payload carries data (16b) and index (16b)
 - Native CSR stream encoding
- Single cycle latency between cores
 - Flow controlled with low buffering
- 24 configurable static routing (colors)
 - Each color has dedicated buffering, is non-blocking
 - All colors are time-multiplexed onto same physical link
- Hardware broadcast/multicast



Designed for Strong Scaling

- Contemporary platforms fail to Strong Scale
- Must spread workload thin
- Eliminate bulk processing
- Fine-grain messaging among cores
- Offload bookkeeping overheads
- Fabric support for communication patterns



Cerebras SDK

A general-purpose parallel-computing platform and API allowing software developers to write custom programs (“kernels”) for Cerebras systems.

Language

CSL: Cerebras Software Language

Host APIs with Python

Libraries

Optimized primitives

Tools

Simulator

Debugger

Visualization

The screenshot displays the Cerebras SDK GUI with an orange header. The main workspace shows a 6x6 grid of processing elements (PEs) with a central 2x2 area highlighted by a black square. To the left, a 'Colors' panel lists '1 x_in', '2 Ax_out', '3 y_out', and '4 b_in' with checkboxes. Below the grid, the 'Instruction Trace' window shows a table with columns: Cycle, OP Addr, OP Name, Dest, and Src0. The 'Source Code' window displays Cerebras Software Language (CSL) code for a task named 'main_task'. The 'Wavelet Trace' window on the right shows a table with columns: Cycle, Color, Ctrl, Link, and Header, and includes a 'Color Filter' and 'Wavelet Format' dropdown.

Cycle	OP Addr	OP Name	Dest	Src0
344	0x3120	s class	0x0 (0x38b7)	0x0 (0x3040)

```
1 var global: i16 = 0;
2 color main_color = 0;
3 color output_color = 1;
4 const dsd = @get_dsd(fabric_color, {fabric_color = output_color, extent = 1});
5 task main_task(wavelet_data: i16) void {
```

Cycle	Color	Ctrl	Link	Header
3	3	0	W	0x0000
1890	3	0	E	0x0000

SDK Components

Layout

CSL layout blocks

Top level CSL layout file with `set_tile_code`, `set_color_config` calls

or

Paint-based layout

More expressive layout engine shared with ML stack

PE Programming

CSL

Define code that runs on PE, e.g. tasks

CSL is also used by internal engineers for some kernels in our ML stack

Runtime

SDK Runtime

Use `memcpy` to copy data to/ from device, launch kernels

Built on top of production HostIO APIs

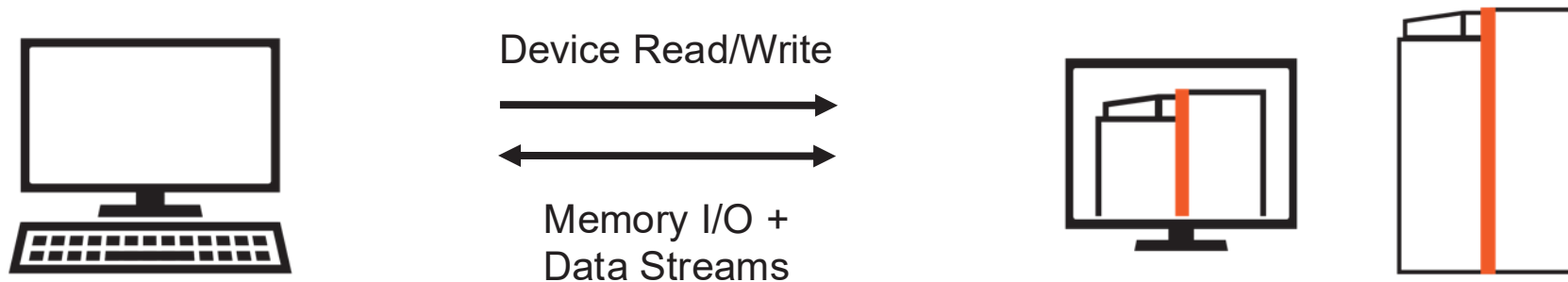
From a Programmer's Perspective

Host CPU(s): Python

- Loads program onto simulator or CS-3 system
- Streams in/out data from one or more workers
- Reads/writes device memory

Device: CSL

- Target software simulator or CS-3
- CSL programs run on groups of cores on the WSE, specified by programmer
- Executes dataflow programs



CSL: Language Basics

- Types
- Functions
- Control structures
- Structs/Unions/Enums
- Comptime

Straight from C
(via Zig)

- Builtins
- Module system
- Params
- Tasks
- Data Structure Descriptors
- Layout specification

CSL specific

**Used for writing
device kernel code**

**Familiar to
C/C++/HPC
programmers**

SDK Example Programs Available

Repository: github.com/Cerebras/csl-examples

- Introductory Tutorials
- GEMV
- GEMM
- Cholesky Decomposition
- 1D and 2D FFT
- 7-Point Stencil SpMV
- Power Method
- Conjugate Gradient
- Preconditioned Conjugate Gradient
- Finite Difference Stencil Computations
- Mandelbrot Set Generator
- Shift-Add Multiplication
- Hypersparse SpMV
- Histogram Computation

SDK Access and Use

The SDK is publicly available for download, with public documentation, examples, and forums

Access and Announcements: cerebras.net/developers/sdk-request

Documentation: sdk.cerebras.net

Forums at discourse.cerebras.net

In addition to offline simulator, SDK jobs can be run on hardware on any appliance cluster, with identical job submission process to ML training jobs

SDK Usage and Impact

The SDK is a public platform for Wafer-Scale Computing supporting external developers and researchers

Scaling the “Memory Wall” for Multi-Dimensional Seismic Processing with Algebraic Compression on Cerebras CS-2 Systems



Hatem Ltaief
Yuxi Hong
Extreme Computing Research
Computer, Electrical, and Mathematical Sciences

Trackable Agent-based Evolution Models at Wafer Scale

Matthew Andres Moreno^{1,2,3,*}, Connor Yang⁴, Emily Dolson^{5,6}, Luis Zaman^{1,2}
¹University of Michigan, Ecology and Evolutionary Biology, Computer Science, and Behavior
²Michigan State University, Computer Science, Evolution, and Behavior
³morenomas@umich.edu

Using Wafer-Scale AI Hardware for Case Study in Developing a Monte Carlo Simulation

Objective

Apply emerging AI/ML hardware accelerators to achieve orders-of-magnitude scale-up of evolutionary computation population sizes, while maintaining diagnostic phylogeny telemetry.

On-device Performance

- 562,500 processors (750x750 WSE array)
- 18 million population size
- 17k generations per second w/ tracking
- tracking cost comparable to evaluation/selection model

Methods

Cerebras Wafer-Scale Engine (WSE) is 850,000 processors, arranged in 2D grid. Highly distributed architecture. Emblematic of emerging AI hardware accelerators.

Apply island-model population structure with asynchronous migration between processors.

Each chip consists of hundreds of thousands of interconnected processing elements (PEs).

Peers connect to neighbors on the same chip. The reader connects to each PE by its four neighbors, allowing it to read and receive data.

Reconstruction Validation

Test correctness of tracking/reconstruction using hardware emulator.

- Genomes are tagged based on their origin in order to measure relatedness between two simulation progresses.
- After neutral evolution, one genome is sampled, then its annotations are used to approximate evolutionary tree.
- Check if genomes from same origin are grouped together.

Future Directions

- Further benchmarking and scaling tests on algorithms in Rust and C++ for use in trade-off studies.

Near-Optimal Wafer-Scale Reduce

Piotr Luczynski
Department of Computer Science
ETH Zurich

Lukas Gianinazzi
Department of Computer Science
ETH Zurich

Patrick Iff
Department of Computer Science
ETH Zurich

Matrix-Free Finite-Volume Kernels on a Dataflow Architecture

Authors: Ryuichi Sai (Rice University); Francois Hamon (TotalEnergies E&P Research and Technology USA, LLC); John Mellor-Crummey (Rice University); and Mauricio Araya-Polo (TotalEnergies E&P Research and Technology USA, LLC)

SPADA: A Spatial Dataflow Architecture Programming Language

Lukas Gianinazzi
Noëda, ETH Zurich
Zurich, Switzerland
lux@noeda.ai

Tal Ben-Nun
Lawrence Livermore National Laboratory
Livermore, California, USA

Torsten Hoefler
Department of Computer Science
ETH Zurich
Zurich, Switzerland

Abstract—Spatial dataflow architectures like the Cerebras Wafer-Scale Engine achieve exceptional performance in AI and scientific applications by leveraging distributed memory across processing elements (PEs) and localized computation. However, programming these architectures remains challenging due to the need for explicit orchestration of data movement through

and various other HPC applications [35, 38, 51, 58]. However, maximizing performance on this architecture necessitates tailoring its unique characteristics. This need for Reduce and AllReduce on the WSE.

Communication Collectives for the Cerebras Wafer-Scale Engine

Automated Code Generation of High-Order Stencils for a Dataflow Architecture

Authors: Ryuichi Sai, John Mellor-Crummey, and Jinfan Xu (Rice University) and Mauricio Araya-Polo (TotalEnergies E&P Research and Technology USA, LLC)

An MLIR Lowering Pipeline for Stencils at Wafer-Scale

Nicolai Stawinoga[†]
Technische Universität Berlin
Berlin, Germany
nicolai@aes.tu-berlin.de

David Katz^{*}
EPCC
The University of Edinburgh
Edinburgh, United Kingdom
d.katz@sms.ed.ac.uk

Anton Lydike
School of Informatics
The University of Edinburgh
Edinburgh, United Kingdom
anton.lydike@ed.ac.uk

Justs Zarins
EPCC
The University of Edinburgh
Edinburgh, United Kingdom
j.zarins@epcc.ac.uk

Nick Brown
EPCC
The University of Edinburgh
Edinburgh, United Kingdom
nick.brown@ed.ac.uk

George Bisbas[†]
Imperial College London
London, United Kingdom
georgios.a.bisbas@gmail.com

Tobias Grosser
University of Cambridge
Cambridge, United Kingdom
tobias.grosser@est.cam.ac.uk

Vyas Giridharan

Anonymous Author

Trackable Agent-based Evolution Models at Wafer Scale

Matthew Andres Moreno^{1,2,3,*}, Connor Yang⁴, Emily Dolson^{5,6}, and Luis Zaman^{1,2}
¹Department of Ecology and Evolutionary Biology, University of Michigan, Ann Arbor, United States
²Center for the Study of Complex Systems, University of Michigan, Ann Arbor, United States
³Michigan Institute for Data Science, University of Michigan, Ann Arbor, United States
⁴Undergraduate Research Opportunities Program, University of Michigan, Ann Arbor, United States
⁵Department of Computer Science and Engineering, Michigan State University, East Lansing, United States
⁶Program in Ecology, Evolution, and Behavior, Michigan State University, East Lansing, United States
*corresponding author: morenomas@umich.edu

Abstract

Continuing improvements in computing hardware are poised to transform capabilities for *in silico* modeling of cross-scale phenomena underlying major open questions in evolutionary biology and artificial life, such as transitions in individuality, eco-evolutionary dynamics, and rare evolutionary events. Emerging ML/AI-oriented hardware accelerators, like the 850,000 processor Cerebras Wafer Scale Engine (WSE), are enabling the simulation of large-scale evolutionary models.

Simulation Runtime

histrat markers

Post-Hoc Analysis

ecology? adaptation? trait origins?

Introduction and Motivation

Simulation of complex systems is a key challenge in many domains. The Cerebras Wafer-Scale Engine (WSE) is a novel hardware accelerator designed for high-performance computing. This paper presents a methodology for simulating large-scale evolutionary models on the WSE, achieving orders-of-magnitude scale-up in population size and simulation time.

Background

The Cerebras Wafer-Scale Engine (WSE) is a novel hardware accelerator designed for high-performance computing. It consists of 850,000 processors arranged in a 2D grid. The WSE is designed to support a wide range of applications, including machine learning, scientific computing, and data processing.

Proposed Methodology

This paper presents a methodology for simulating large-scale evolutionary models on the WSE. The methodology is based on the use of a dataflow architecture, which allows for the efficient execution of large-scale simulations. The methodology is designed to be scalable and flexible, allowing for the simulation of a wide range of evolutionary models.

Experimental Results

The experimental results show that the proposed methodology achieves orders-of-magnitude scale-up in population size and simulation time compared to traditional simulation methods. The results also show that the methodology is scalable and flexible, allowing for the simulation of a wide range of evolutionary models.

Results and Analysis

The results of the simulation show that the proposed methodology is able to simulate large-scale evolutionary models efficiently. The results also show that the methodology is able to capture the complex dynamics of these models, providing valuable insights into the evolution of complex systems.

E-2: Evaluating

Filip Dobrosavljević
dofilip@student.ethz.ch
ETH Zurich
Switzerland

Torsten Hoefler
torsten.hoefler@inf.ethz.ch
ETH Zurich
Switzerland

Pre-order Reduce

1D Allreduce

2D

Results

Performance Evaluation

Grid-COO
Grid-CSC
Grid-CIR
Grid-Elpack

Systems Inc. All Rights Reserved